



Intelligent Robotics “Contract” presentation

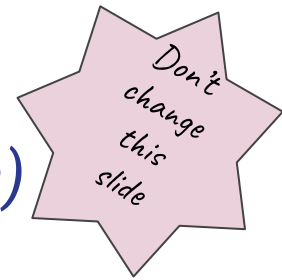
Armando Sousa & Jorge Ribeiro & Maria Silva Lopes

Explanation & Template slides follows...



Group $G\{N\}$

don't change this slide (explanation)



Copy the template slides:

Slide 1 - List of Students + ⁽ⁱ⁾ Project title + General Objectives;

Slide 2 - ⁽ⁱⁱ⁾ Intermediate GOALS;

Slide 3 - ⁽ⁱⁱⁱ⁾ Goals for Assignment and ^(iv) Extra merit goals / issues;

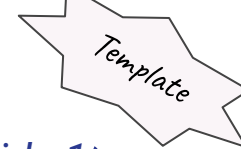
Slide 4 - ^(v) Tools to be used + ^(vi) limitations noted (if any);

(...later, at interm. present. time...) + Slide 5 - evidence that interm. GOALS have been met

Obs: 4..5 slides per group, keep the order of the groups,
delete example and change group_ID,
=> plan for presentation time of **7 min** max

Group G{N} - Formal & Title & Goals

don't change this slide (TEMPLATE slide 1)



- Team Members:
 - Name <upnumber@up.pt>
 -
 -
 -
- Title and General Objectives: “Example Covid sanitizer robot”
 - Example Moving robot that sprays sanitizer
 - Example Moving robot with UV lamp
 - Example Work-Life Balance
 - Example Cure cancer

Group $G\{N\}$ - Intermediate Goals

don't change this slide (TEMPLATE^{slide 2})



- Intermediate Goals:
 - Example Get robot running
 - Example Buy sanitizer and lamps
 - Example Buy face masks



Group G{N} - Course Project (Final) Goals

Template

don't change this slide (TEMPLATE^{Slide 3})

- Main Goals:
 - Example SoA search
 - Example Get robot running, actually spraying and cleaning with UV light
 - ...
- Extra Merit:
 - Example Follow waypoints
 - Example Measure spray and Light at 2 meters distance

Group G{N} - Tools & Limitations

Template

don't change this slide (TEMPLATE Slide 4)

- Tools:
 - Example Robot
 - Example Sanitizer
 - Example UV Lights
 - Example Stores that really have inventory
- Limitations:
 - Will not actually cure infected people

Group G{N} - Evidence of Intermediate Goals

Template

don't change this slide (TEMPLATE Slide 5)

- Architecture ...
- Difficulties...
- Do a 7-min oral intermediate presentation
- In the same slides of the contract, add an extra slide or two for evidence that intermediate goals have been met in accordance to contract - example: 20s video...
- Prepare to answer questions such as
 - "all tools validated?",
 - "next steps?",
 - "expected difficulties?", etc.
- Prepare to explain division of tasks inside the group
- The contract is deemed stable / "sealed" after intermediate presentation

Slide(s) to be used
at the intermediate
presentation

Have fun and prepare all deliverables from the very first moment!

Group G{N} - Evidence of Intermediate Goals

Template

don't change this slide (TEMPLATE Slide 5)

- Example: Selfie of the group with the sanitizer robot

Slide(s) to be used
at the intermediate
presentation



Groups...

Please insert the slides for your group in the correct sequence ;)

Class 1 MIA01

Group G17 - Formal & Title & Goals

- Team Members:
 - Diogo Ferreira <up202205295@up.pt>
 - Daniela Osório <up202208679@up.pt>
 - Pabline Nascimento <up202512028@up.pt>
 - Marcella Duque <up202512342@up.pt>
- Title and General Objectives - Simplified Robot Soccer (1v1, RL-based)
 - Develop a simplified robot soccer simulation with a final 1v1 scenario
 - Train a RL-controlled robot to shoot and control the ball
 - Implement decision-making in a dynamic environment with an opponent
 - Evaluate learning performance and gameplay success over time

Group G17 - Intermediate Goals

- Intermediate Goals:
 - Build basic 1v0 soccer simulation environment: one robot, one ball, one goal
 - Implement omnidirectional robot movement and ball dynamics
 - Define the RL state/observations, actions, and reward function
 - Train an initial RL agent to approach the ball and attempt shots
 - Visualize gameplay in Webots and monitor training performance

Group G17 - Course Project (Final) Goals

- Main Goals:
 - Train an RL-controlled robot to play a final 1x1 soccer scenario
 - Achieve goal scoring, ball control and basic opponent avoidance
 - Evaluate performance against a baseline opponent, such as a random or scripted agent
- Extra Merit:
 - Compare different RL algorithms or reward strategies
 - Add dynamic ball scenarios where the ball starts with random velocity and the robot must intercept it before shooting
 - Extend to a 2v2 setting with team coordination and communication, only if the 1v1 goal is completed early

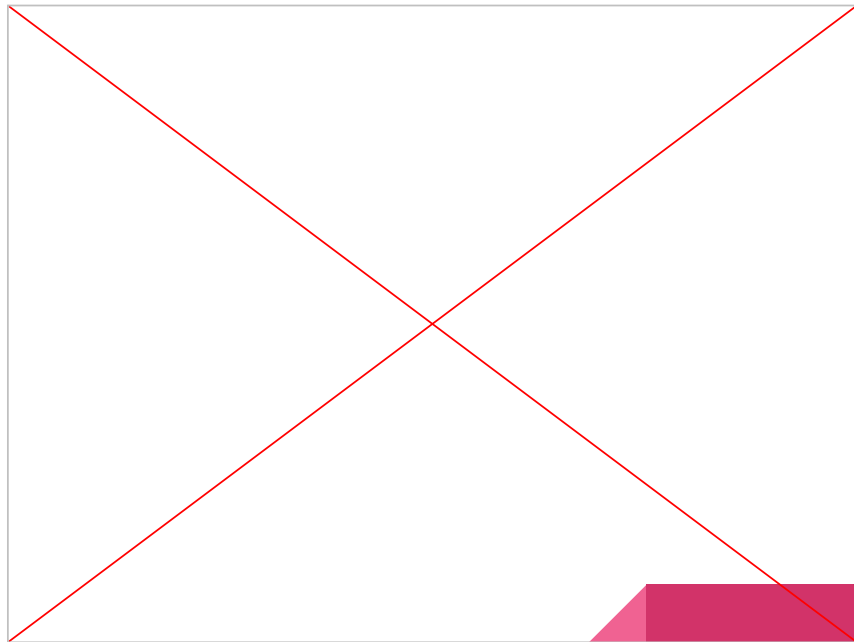
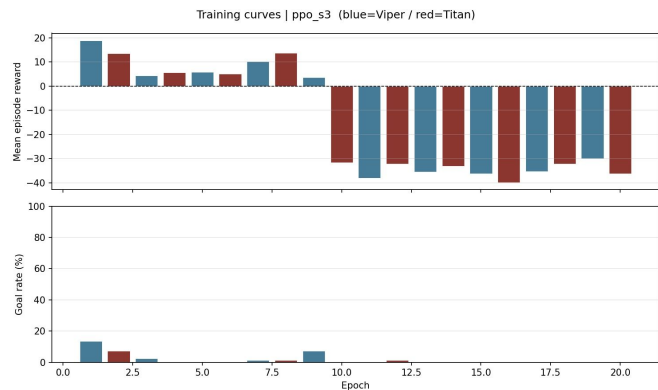
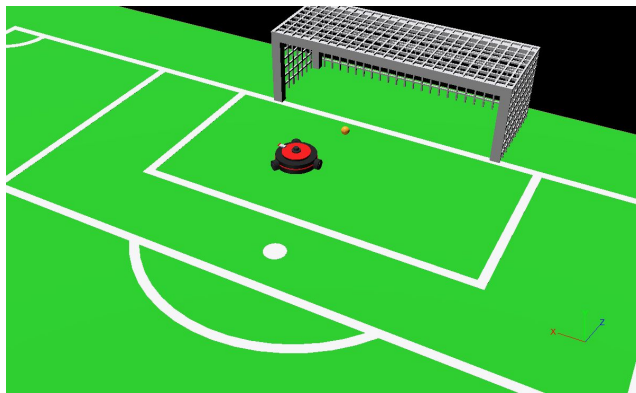
Group G17 - Tools & Limitations

- Tools:
 - Python
 - Webots
 - StableBaseline3 + Gymnasium
 - Pytorch
 - Matplotlib / TensorBoard
 - NumPy / pandas
- Limitations:
 - Limited training time may affect agent performance
 - Simplified physics and environment compared to real robot soccer
 - Simulation only, with no real robot deployment
 - Focus on basic behaviors , not advanced strategies or team coordination

Group G17 - Evidence of Intermediate Goals

- Architecture
 - Webots world
 - Python Supervisor
 - Gymnasium Environment
 - PPO Agent
- Intermediate Goal Achieved
 - 1v0 setup with one robot, one ball and one goal
- Implemented
 - Robot Movement, ball dynamics, observations, actions and reward Function
- Difficulties
 - Reward Tuning
 - Training Time
 - Stable ball control

Group G17 - Evidence of Intermediate Goals



Group G18 - Formal & Title & Goals

- Team Members:
 - Guilherme Oliveira <up202204987@up.pt>
 - Magda Costa <up202207036@up.pt>
- Title and General Objectives: **The Perfect Heist: Planning and Executing Safe Navigation with Reinforcement Learning**
 - Design an autonomous agent capable of performing a stealth mission
 - Plan the infiltration route before execution using classical path planning
 - Learn to execute the plan intelligently using Reinforcement Learning
 - Avoid detection by a surveillance guard with limited field of view
 - Achieve safe and successful mission completion

Group G18 - Intermediate Goals

- Intermediate Goals:
 - Create a simulation environment with map layout, obstacles, target, and a rotating guard with limited field of view
 - Implement A* to compute a complete route (start → target → exit)
 - Enable waypoint-based navigation without learning
 - Implement realistic detection using field of view and occlusion (line of sight)
 - Integrate Reinforcement Learning to control actions (move, wait, adjust direction)
 - Train the agent to minimize detection risk and complete the mission

Group G18 - Course Project (Final) Goals

- Main Goals:

- Autonomous agent successfully completes the mission (reach the target and escape without being caught)
- Agent follows a route planned before execution (A* path planning)
- Agent uses Reinforcement Learning to make execution decisions (move, wait, adjust)
- Agent avoids detection by a guard with field of view and occlusion
- Agent demonstrates intelligent behavior under risk (timing, avoidance, adaptation)

- Extra Merit:

- Add guard patrol routes
- Add guard investigation of last suspected position
- Allow the agent to choose between multiple planned routes based on risk

Group G18 - Tools & Limitations

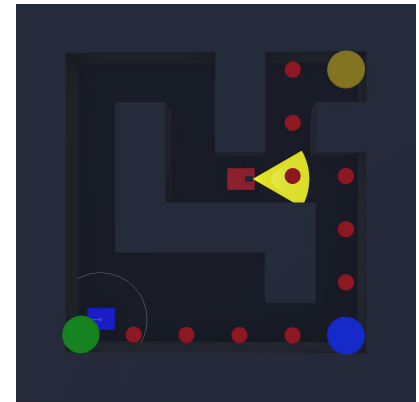
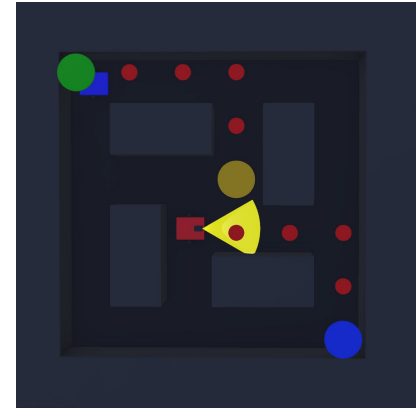
- Tools:

- Webots (robotics simulation environment)
- Python (programming language)
- Gymnasium (Reinforcement Learning environment interface)
- Stable-Baselines3 (PPO implementation)
- Matplotlib / TensorBoard (visualization and training analysis)

- Limitations:

- Static guard behavior in the base scenario (no movement)
- Known environment (no map exploration required)
- Simplified perception model (field of view and line of sight instead of vision-based detection)
- Performance depends on reward design and training stability

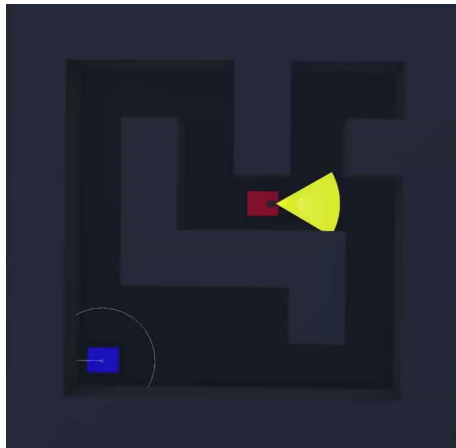
Group G18 - Evidence of Intermediate Goals



Group G18 - Evidence of Intermediate Goal

Navigation rewards

- Time penalty
- Progress reward toward current waypoint
- Waypoint reward
- Treasure reward
- Exit reward
- Alignment reward
- Forward-motion reward
- LiDAR / collision / timeout penalties



Light rewards

- Light exposure penalty
- Light escape progress reward
- Directional light escape reward
- Contextual wait/flee reward
- Alert and detected penalties
- Navigation reward scaled down while in light



What we tried for light

Curriculum learning: start from the navigation model and fine-tune for light.

Training from scratch directly in the light phase.

Reward shaping with wait vs flee modes, escape progress, directional hints and detection penalties.

Current hypothesis

The light-phase reward may be competing with navigation instead of cleanly improving it.

We need to inspect metrics to identify reward hacking, excessive penalties, or weak escape signals.

Next step: tune the reward balance and validate behavior from episode metrics and videos.

Group G19 - Formal & Title & Goals

- Team members:
 - Eduardo José Pereira Matos Passos <up202205630@up.pt>
 - Gabriel da Costa Filipe Carvalho <up202208939@up.pt>
 - Guilherme Moura Pinho Andrade Silva <up202205298@up.pt>
- Object-Level Semantic Mapping via Offline LiDAR-Camera Fusion
 - Project 2D detections into 3D space using extrinsic calibration and SLAM poses.
 - Record synchronized camera, LiDAR and pose streams to for repeatable offline processing.
 - Aggregate repeated detections of the same physical object across frames into stable, persistent map entities.
 - Display the object map interactively and intuitively.

Group G19 - Intermediate Goals

- Intermediate goals
 - Review SoTA literature on 3D scene graphs: Hydra, ConceptGraphs, S-Graphs, CURB-SG
 - ROS 2 running on Jetson Orion Nano; stable teleop of the Waveshare rover
 - Drive the rover through a test space; record RGB-D, LiDAR, odometry and TF to a ROS 2 bag
 - YOLOv8 / Grounding DINO running on RGB frames, producing 2D detection with class+confidence
 - RGB-D intrinsics, camera-to-LiDAR and camera-to-base extrinsics validated on a calibration target
 - End-to-end run: bag -> 2D detections -> back-projected 3D bounding boxes, viewable in Open3D

Group G19 - Course Project (Final) Goals

- Main goals

- Projection module that correctly back-projects 2D bounding boxes into 3D space, with timestamp synchronization between sensors, resolving repeated detections of the same object across frames into a single entity
- Final semantic map of a simple indoor scene with at least 5 distinct object classes correctly localized and labeled
- Visualization showing the objects in 3D space with their labels, confidence, and accumulated point clouds

- Extra Merit

- Repeated observations labels' probability fusion to improve label certainty
- CLIP embeddings from the detected 2D crops to disambiguate close objects of the same class

Group G19 - Tools & Limitations

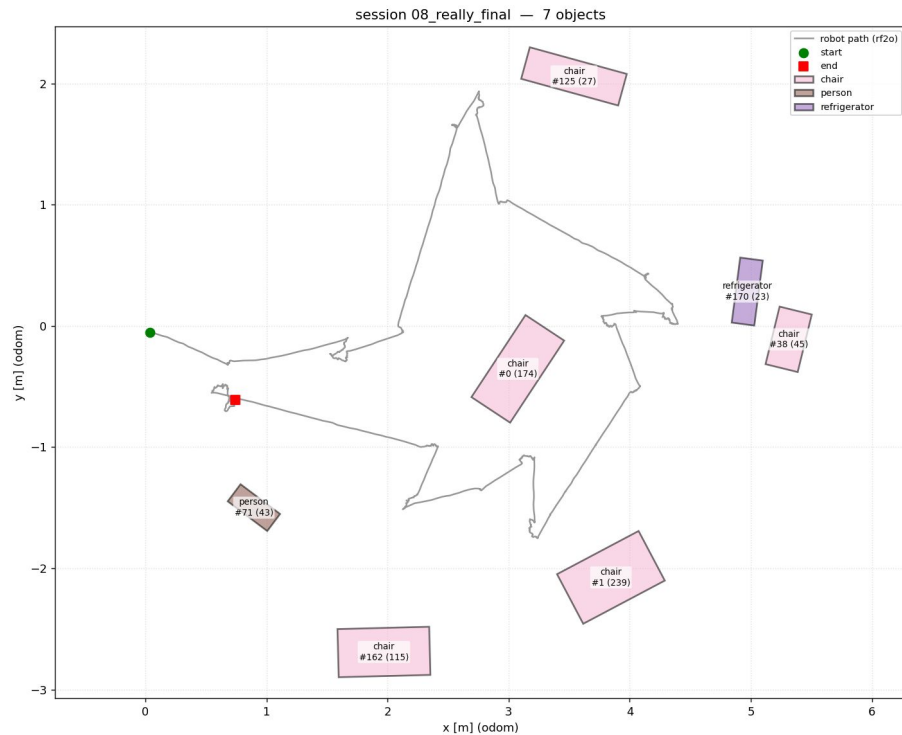
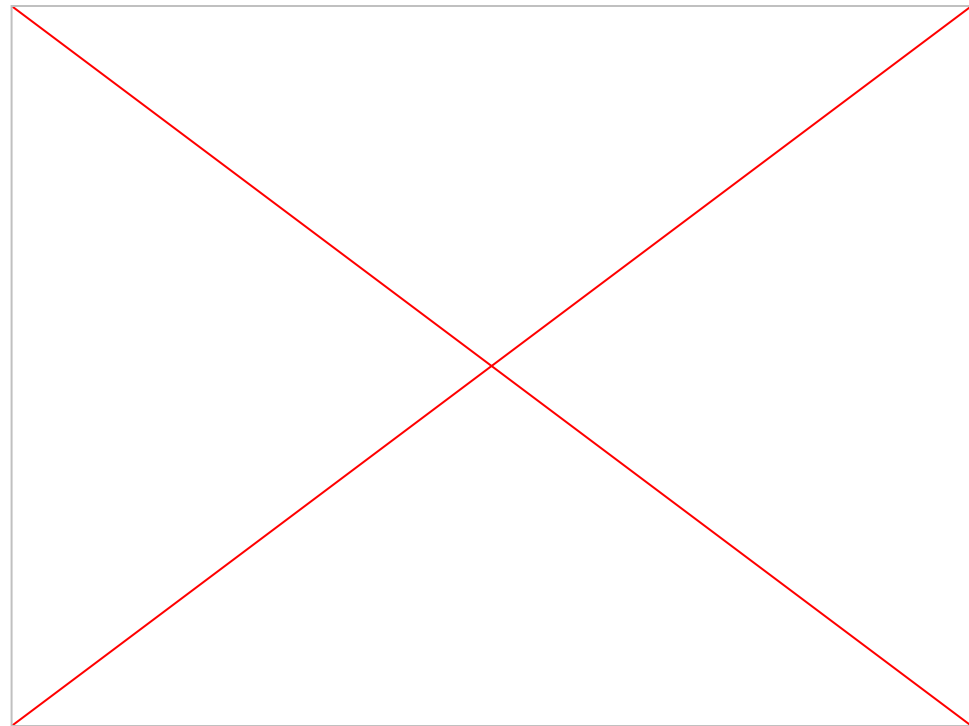
- Tools

- **Hardware:** Waveshare UGV Rover (Jetson Orion Nano) - laptop for offline processing
- **Software:** ROS 2 Humble/Jazzy - ROS2 bag for data collection - Open3D for 3D visualization - PyTorch + YOLOv8/Grounding Dino - Python
- **Workflow:** Git - LaTeX (article) - indoor recording sites (apt. + FEUP room)

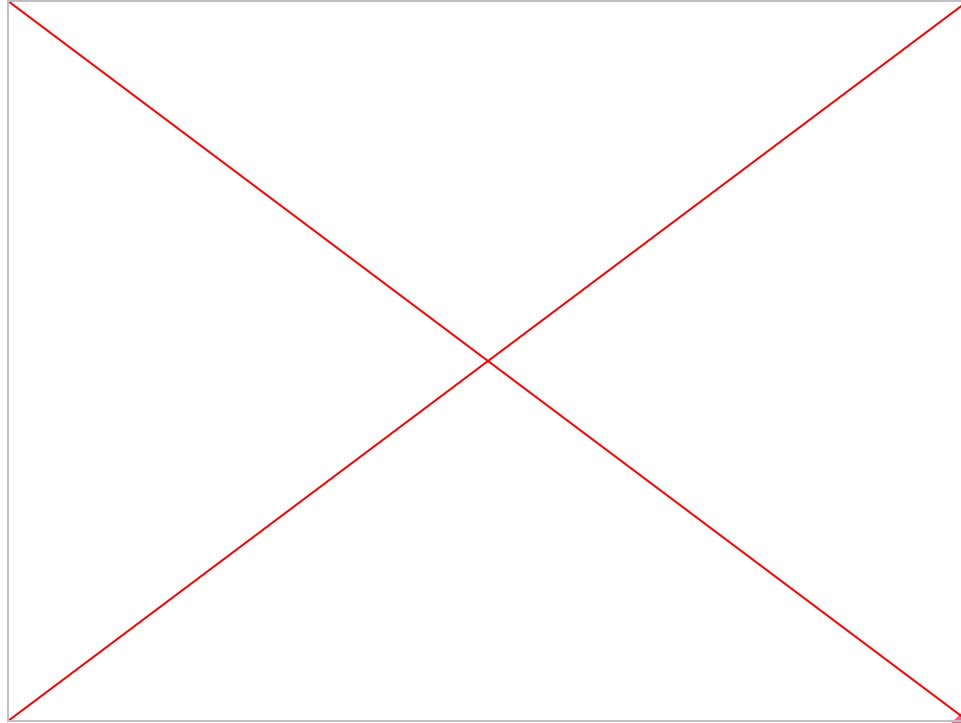
- Limitations

- Offline processing only; no real-time mapping
- Indoor, static scenes
- Single robot and single bag per experiment, no cooperative mapping
- Object classes are limited by the chosen detector

Group G19 - Evidence of Intermediate Goals



Group G19 - Evidence of Intermediate Goals



Group G20 - Formal & Title & Goals

- Team Members:
 - Francisco Tavares <up202205119@up.pt>
 - Rodrigo Batista <up202206259@up.pt>
 - Rodrigo Taveira <up202206425@up.pt>
- Title and General Objectives: “Reinforcement Learning for Leader-Follower Robot Control in Webots”
 - Learn optimal pursuit trajectories using DRL (PPO/DQN).
 - Navigate safely from point A to B using obstacle-aware heuristics.
 - Transition from relative position inputs to raw 2D LiDAR scanning.
 - Evaluate performance in environments with static and moving obstacles.

Group G20 - Intermediate Goals

- Intermediate Goals:
 - Full setup of the Webots environment with TurtleBot3 models.
 - Implementation of a robust heuristic navigation system (A^* with smoothing for less points) for the Leader to travel from point A to B while avoiding static walls.
 - Successful training of the Follower agent to follow the target using relative position coordinates as input.
 - Definition and validation of a reward system that balances distance to the leader, collision avoidance, and path efficiency.

Group G20 - Course Project (Final) Goals

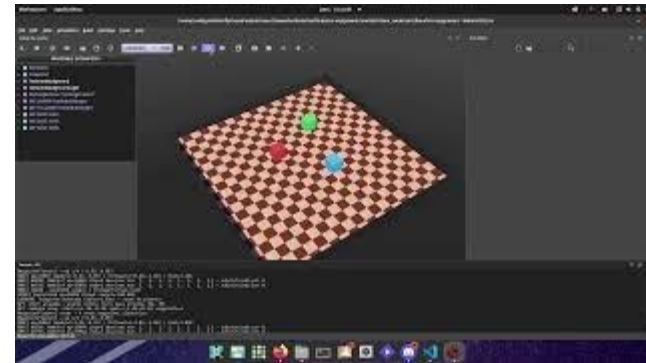
- Main Goals:
 - Transitioning the "Police" agent's input from global coordinates to raw 2D LiDAR data, requiring the model to learn target identification from point clouds (already in progress).
 - Testing the pursuit performance in environments with moving obstacles that interfere with the LiDAR line-of-sight.
 - Quantitative analysis comparing the RL-based "Police" against a baseline heuristic pursuit algorithm (e.g., pure pursuit).
- Extra Merit:
 - Implementing a two-Police swarm using centralized or cooperative learning to surround and trap the fugitive.

Group G20 - Tools & Limitations

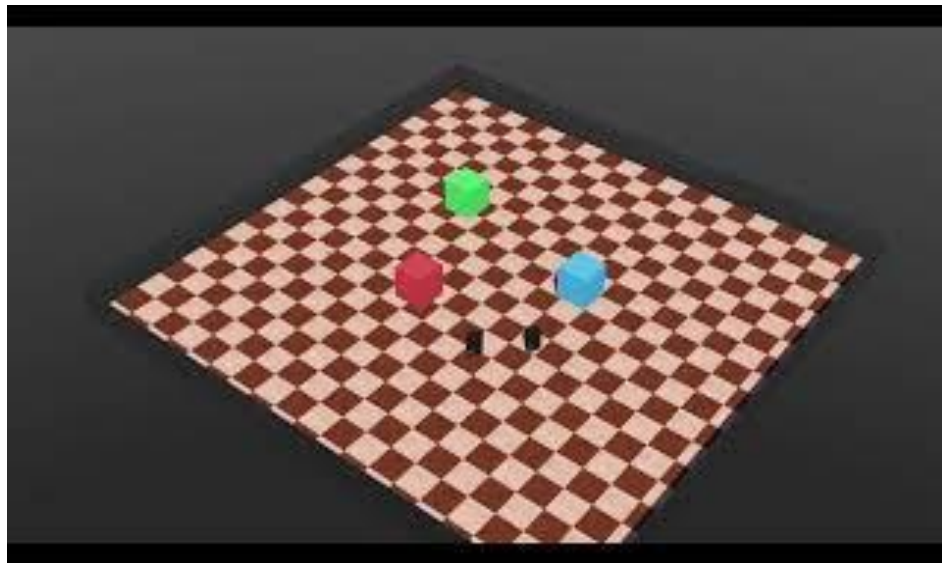
- Tools:
 - TurtleBot3
 - Webots
 - Gymnasium
 - StableBaselines3
 - Python
- Limitations:
 - Limited training time
 - Reality Gap (only trained on simulation)

Group G20 - Evidence of Intermediate Goals

- Architecture
 - **Robot Leader**
 - Input: Map grid with obstacles, start and end positions
 - Extra: Smoothing of points
 - **Robot Follower**
 - Observation space (phase 1): Raw LiDAR (8 rays), Dx and Dy of leader robot
 - Action Space: linear velocity and angular velocity
- Difficulties
 - Reward stabilizing too soon;
 - Implementation of the supervisor in Webots;
 - Fast timesteps on Webots lead to unstable performance.



Group G20 - Evidence of Intermediate Goals



- Future work
 - Remove relative positions from the Observation Space, while increasing LiDAR rays (with CNN for processing)
 - Different scenarios;
 - Different accelerations for the leader robot;
 - Implement metrics;
 - Add one more follower.

Group G21 - Formal & Title & Goals

- Team members:
 - João Lamas – up202208948@up.pt
 - Valentina Cadime – up202206262@up.pt
 - Vasco Melo – up202207564@up.pt
- Reinforcement Learning for Autonomous Racing
 - Develop a Reinforcement Learning (RL) agent capable of navigating complex F1 racing tracks in Webots using purely reactive, mapless navigation (LiDAR and car telemetry);
 - Implement a progressive Curriculum Learning pipeline: training the agent on isolated difficult track segments before transitioning to full-lap navigation;
 - Evaluate the computational efficiency and convergence speed of the Curriculum Learning approach against a standard end-to-end full-track training baseline.

Group G21 - Intermediate Goals

- **Simulation Environment:** Complete Webots environment setup with a functional vehicle model and a primary baseline track (Austin F1 Circuit);
- **Gymnasium Interface:** Successful bridge between Webots and Stable-Baselines3 (SB3) ensuring the car receives steering/throttle commands;
- **Sensor Validation:** Successful telemetry extraction including speed, orientation/steering, and LiDAR point clouds;
- **First Lap:** The RL agent must complete one full lap on the baseline track without crashing, while only learning on short difficult segments.

Group G21 - Course Project (Final) Goals

- Main goals
 - **Hierarchical Training:**
 1. *Phase 1: Training on progressively larger track segments (short corners up to full sectors) using a survival-focused reward to efficiently establish basic vehicle control;*
 2. *Phase 2: Transitioning this agent to full-lap navigation, tuning the reward structure to maximise speed and encourage the discovery of faster/better racing lines.*
 - **Implicit Sensor Fusion:** Integration of different sensor readings (velocity, orientation, LiDAR) to directly train the RL Agent;
 - **Comparative Analysis:** A study comparing the success rate, convergence speed and lap times of a model trained via the Curriculum approach versus a model trained end-to-end on the full track from scratch.
- Extra Merit / Issues
 - **Zero-Shot Transfer Learning:** Evaluating the agent's ability to navigate an unseen track with zero additional training;
 - **Domain Randomisation:** Introducing controlled noise to sensor readings and actuator responses.

Group G21 - Tools & Limitations

- Tools

- **Simulator:** Webots (for 3D physics and realistic tire friction);
- **RL Framework:** Stable-Baselines3 (SAC);
- **Environment API:** Gymnasium (Python bridge);
- **Track & Architecture Inspiration:** F1tenth Autonomous Racing.

- Limitations

- **Hardware Constraints:** Training deep RL models on local machines (Mac/Linux) may limit the number of iterations or total training time;
- **Physics Gap:** While Webots is realistic, it is not a 100% match for real-world racing dynamics (Sim-to-Real gap);
- **Reward Function Design:** Minor flaws in the reward structure can drastically impact convergence speed, as well as lead to reward exploitation and sub-optimal driving policies.

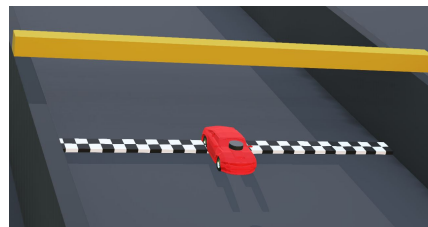
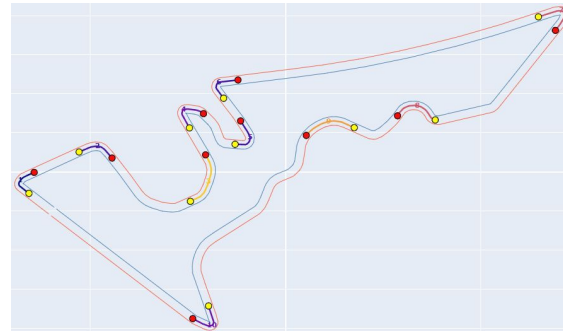
Group G21 - Evidence of Intermediate Goals

- Architecture Pipeline

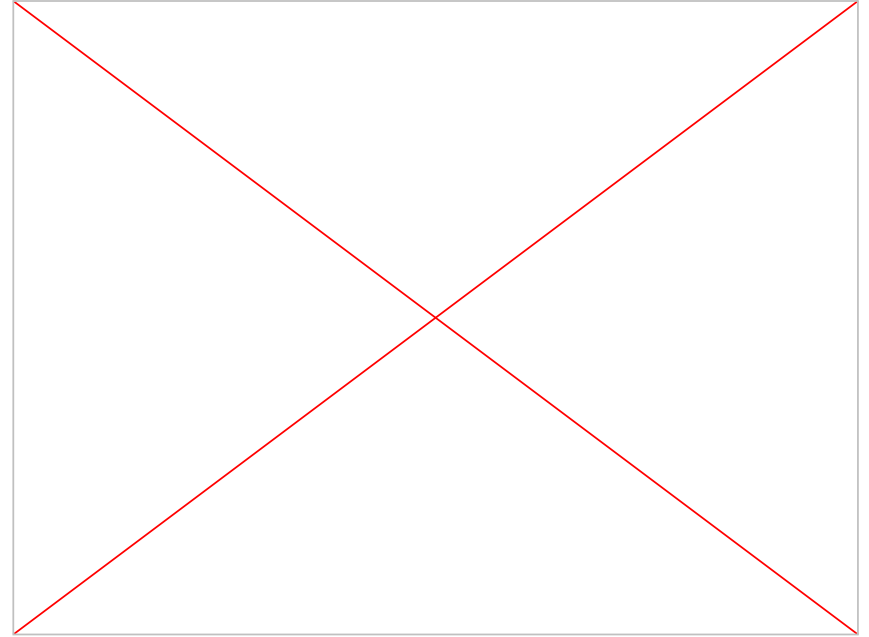
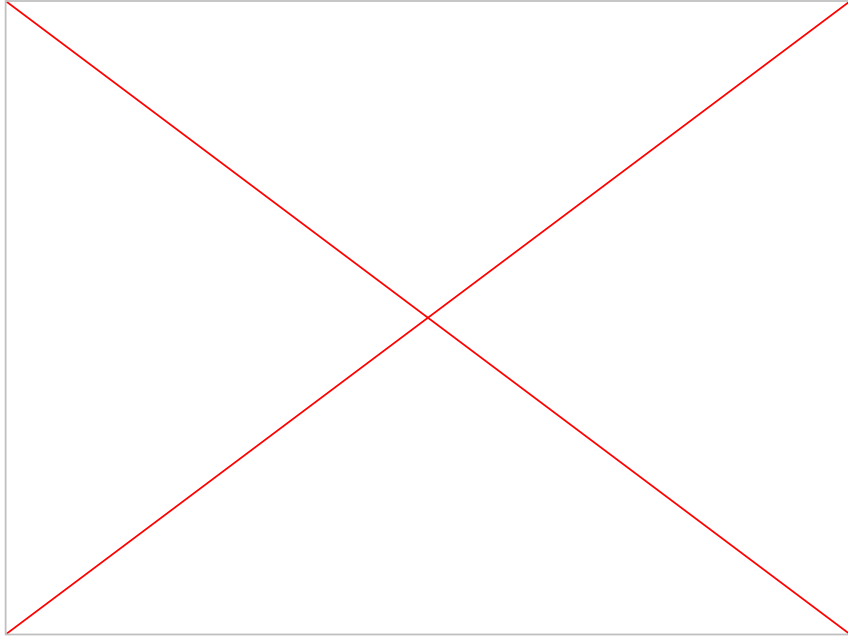
- **Input:** Mapless LiDAR array and real-time vehicle telemetry;
- **Time Awareness:** Frame-stacking multiple timestamps to capture motion and optical flow;
- **Processing:** 1D CNN extracts spatial features from raw sensor data;
- **Outputs:** Continuous steering and throttle commands.

- Potential Difficulties

- **Compute Bottlenecks:** Heavy simulation demands Slowing down training runs;
- **Reward Exploitation:** The AI finding “lazy” survival loopholes;
- **Track Overfitting:** The agent memorising specific corners Instead of learning to drive.



Group G21 - Evidence of Intermediate Goals



Group G23 - Formal & Title & Goals

- Team members:
 - Alexandre Sousa <up202206427@up.pt>
 - Ricardo Costa <up202107547@up.pt>
 - Sara Táboas <up202205101@up.pt>
- Title and General Objectives - **Intelligent system for Multi-Robot Task Scheduling and Allocation with Reinforcement Learning**
 - Design an intelligent system capable of effectively assigning tasks to multiple robots using reinforcement learning, including:
 - **Motion planning** (heuristic-based)
 - **Task allocation** (trained reinforcement learning)

Group G23 - Intermediate Goals

- Intermediate Goals:
 - **Problem definition and Representation:** Robots definition (location and pose, availability and current state,), Tasks definition (location, priority, duration, current state)
 - **Graph Neural Network definition:** Nodes (robots + tasks + locations) and Edges (time, distance, cost)
 - **Train Baseline Model:** Train the GNN with a simple heuristic function
 - **Reinforcement Learning definition:** Reward function definition (using a reward for completed task and penalizations – time, distance, etc.); PPO algorithm definition and Action definition (e.g.: action = (robot, task))

Group G23 - Course Project (Final) Goals

- Main Goals:
 - **Baseline Simulation:** Create an abstract simulation to visualize the task allocation through the robots
 - **Train GNN with RL:** Compare the use of Reinforcement Learning with heuristic algorithms
 - Allocate the Task with a Graph Neural Network + Reinforcement Learning
 - **Motion Planning:** Use heuristic algorithms to plan the motion of robots.
- Extra Merit:
 - Dependent tasks and multiple robot tasks
 - Develop a more accurate simulation with Gazebo/Webots

Group G23 - Tools & Limitations

- Tools:

- System definition and modeling: **Python, NumPy**
- Environment/Simulation development: **Python, Gazebo/Webots** (extra merit)
- Graph Representation (GNN): **PyTorch / PyTorch Geometric**
- Reinforcement Learning: **Stable-Baselines3**

- Limitations:

- Difficulty in integrating a realistic simulator for the problem.
- Defining a number of robots, tasks and their duration enough to make the Reinforcement Learning work and compare it with classic algorithms. Adjust the training to the task.
- Defining a good state representation a reward function.

Group G23 - Evidence of Intermediate Goals

Architecture

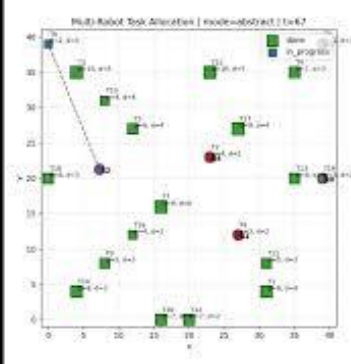
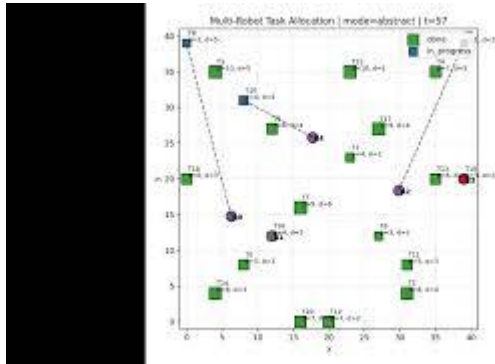
- Initial simulations with simple heuristics (distance-only, priority-only, distance-priority)
- Datasets with Graph instances (from random scenarios)
- Baseline GNN trained on the distance-priority heuristic (92% accuracy on test)
- GNN + Genetic Algorithm (46% accuracy on test)
- RL Definition:
 - **State** → graph representation of the current allocation state
 - **Action** → edge_id (corresponds to robot_id–task_id pair)
 - **Reward** → !!!!!!!
 - **Algorithm** → PPO

Group G23 - Evidence of Intermediate Goals

Difficulties

- Current complexity of the environment (2-5 robots, 6-20 tasks) → Realistic?
 - If complexity == too high → RL training scales up
 - If complexity == too low → GNN (heuristic) too easy

Simulations (GNN heuristic and GA, respectively)



Class 1 MIA02

Group G11 - Formal & Title & Goals

- Team members:
 - António Romão <up202108704@up.pt >
 - Pedro Leitão <up202107852@up.pt>
- Title and General objectives - Robo-C.O.P - A **C**ompetitive **O**bject **P**ushing **G**ame
 - Implement a 3v3 game, where two teams of 3 RL automated robots fight for objects in a map. Whoever gets more blocks in their base, wins..
 - Solve the multi-robot pushing problem, in which 2 or more robots need to work together to push heavier objects.

Group G11 - Intermediate Goals

- Intermediate goals
 - Develop the Webots simulation with the proper map and robots. (placing the objects at random, robot base areas, map design etc.)
 - Implement the object pushing reliably.
 - Honest perception: line-of-sight only (LIDAR + odometry).
 - Setup the MARL (Multi-Agent Reinforcement Learning).
- Note
 - Simulator changed from Webots → PyMunk (~100× faster, full collision control)
 - Same RL stack: Gymnasium + Stable-Baselines3 (PPO).

Group G11 - Course Project (Final) Goals

- Main goals
 - Achieve proper games on 3v3 settings.
 - Evaluate the performance against a competing random team (baseline).
 - Achieve agents that can reliably score more points than a competing autonomous team.
 - Discover emerging/winning strategies (do agents learn to steal from the opposing team? or to block enemy paths? try to be as fast as possible? more or less information sharing? etc.).
- Extra Merit
 - Implement a dynamic map (moving walls).
 - More teams (3v3v3).
 - Limited base space.
 - Compare multiple RL algorithms (PPO vs. MAPPO vs. SAC).

Group G11 - Tools & Limitations

- Tools

- PyMunk (2D rigid-body physics - replaced Webots).
- OpenAI Gymnasium (environment interface).
- Python 3.11 + Pygame (visualisation).
- StableBaselines3 / PPO (RL training).

- Limitations

- Performance (2v2 or 3v3 depends on this).
- Time to train for multi-agent self-play.
- Static environment (agents may need retraining on different maps).

Group G11 - Evidence of Intermediate Goals

- Changed / Updated
 - **Simulator migration:** Webots → PyMunk — $\sim 100\times$ faster training, wall-penetration bug fixed, full physics control.
 - Same RL stack kept (Gymnasium + SB3/PP0).
- Achieved
 - **Reliable object pushing:** correct collision physics validated in randomised maps.
 - **Robust navigation:** single robot navigates any map layout without getting stuck.
 - **1v1 self-play working:** one shared policy controls both robots, learns by playing against itself.
 - **Honest perception validated:** green line = sees block, orange = sees rival, lines disappear behind walls — no privileged position data at inference.

Group G11 - Evidence of Intermediate Goals



Group G13 - Formal & Title & Goals

- Team Members:
 - José Guilherme Santos <up202208081@up.pt>
 - Mateus Cabral <up202511402@up.pt>
 - Victor Rodriguez <up202105114@up.pt>
- Title and General Objectives - Implement SLAM in UGV Rover
 - Implement SLAM on a real-world Waveshare UGV Rover
 - Generate the maps based on SLAM



Group G13 - Intermediate Goals

- Intermediate Goals:
 - Research Simulation environment provided by the Waveshare team
 - Setup all the necessary topics for locomotion and SLAM in ROS2
 - Research and implement SLAM

Group G13 - Course Project (Final) Goals

- Main Goals:
 - Implement SLAM on a real-world Waveshare UGV Rover
 - Generate the maps based on SLAM
- Extra Merit:
 - Implement more SLAM approaches
 - Implement goal-driven autonomous navigation

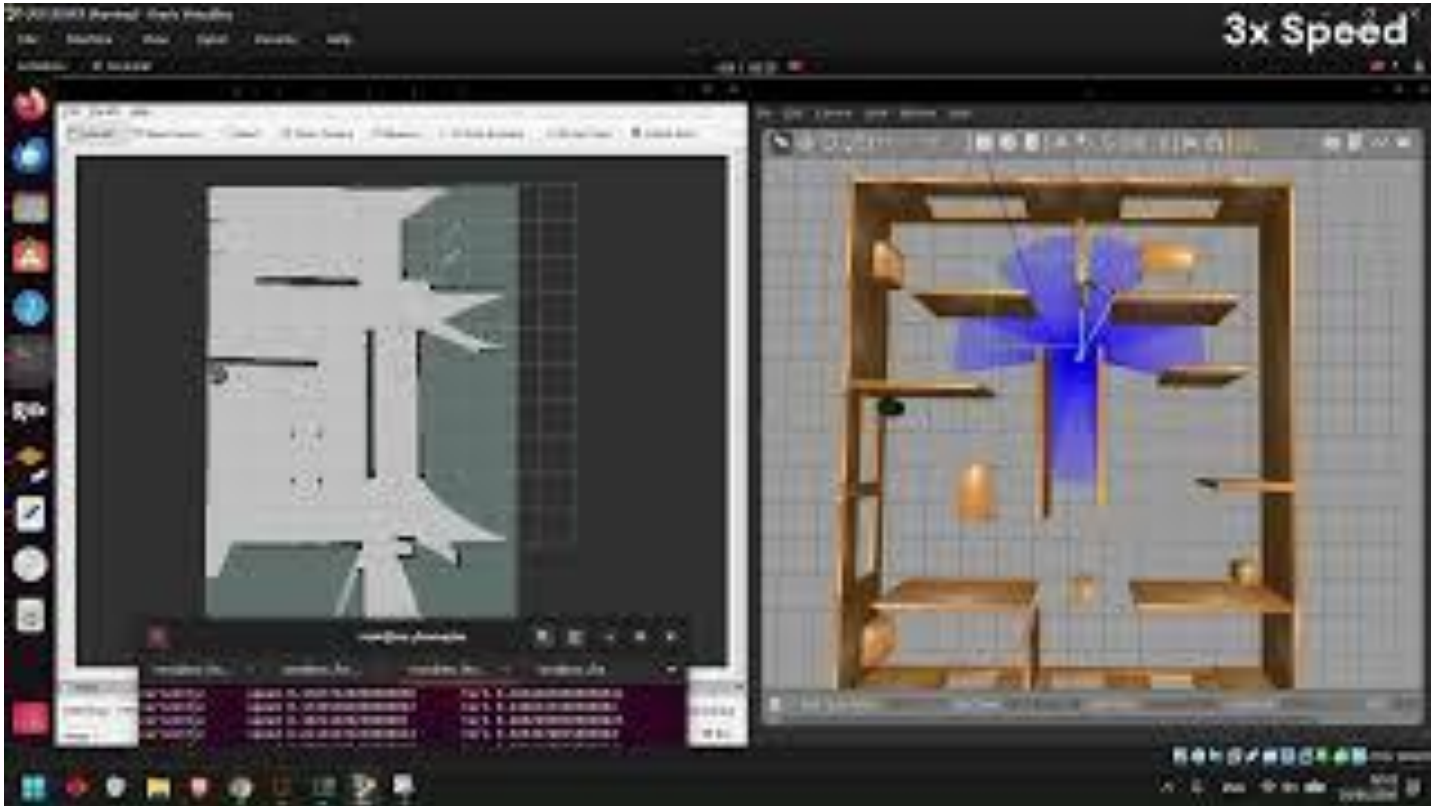
Group G13 - Tools & Limitations

- Tools:
 - Waveshare UGV Rover (Jetson Orion Nano)
 - ROS 2
 - Gazebo
- Limitations:
 - Sensitive to challenging environments
 - Simulation may not match real-world behavior

Group G13 - Evidence of Intermediate Goals

- What's been done:
 - Utilized the simulator provided by the waveshare team
 - Experimented with SLAM already implemented in the simulator (cartographer)
 - Implemented a new SLAM (slam_toolbox, behaves better with explore_lite)
 - Explored ugv locomotion methods already implemented (nav2 and keyboard controls)
- What needs to be done:
 - Validate the SLAM in the real robot (not simulated)
 - Import the maps
 - Explore a bit more goal driven navigation (nav2)

Group G13 - Evidence of Intermediate Goals



Group G22 - Formal & Title & Goals

- Team Members:
 - Alexandre Costa <up202207499@up.pt>
 - Afonso Castro <up202208026@up.pt>
 - Rafael Novais <up202208591@up.pt>
- DeepFight: RL for Autonomous Robotic Striking in a Simulated Combat Environment
 - Build an RL-powered humanoid robot capable of dynamic striking within a physics-based simulation.
 - Apply Deep Reinforcement Learning (DRL) techniques to allow the agent to self-discover combat strategies.
 - Observe and document how complex traits, such as spatial adaptation, timing, and defensive tactics, naturally evolve under varying training conditions.

Group G22 - Intermediate Goals

- Intermediate Goals:
 - **Problem Definition and Environment Setup:** Configure the physics engine and build the humanoid models.
 - **Reward Function Design:** Craft the mathematics to balance striking force with physical stability, given the constraints of a single arm.
 - **First Learning Agent:** Train the baseline agent using one arm against a fixed target, then progress to intercepting a moving target, on different spawn locations.
 - **Evaluation metrics (win rate, time, etc.):** Track these combat stats to actively measure the emergence of adaptation, timing, and strategy.
 - **Compare algorithms:** Test different approaches to determine which processes continuous physical combat most efficiently.

Group G22 - Intermediate Presentation



Group G22 - Course Project (Final) Goals

- Main Goals:
 - **Efficient Striking:** The agent learns to hit available targets efficiently under physical and movement restrictions.
 - **Dynamic Adaptation:** The agent successfully adapts its timing and trajectory to hit both randomly positioned and moving targets.
 - **Range Management:** Targets spawn outside the robot's immediate reach, forcing the agent to coordinate lower-body movement with upper-body striking.
- Extra Merit:
 - **Defensive Mechanics:** The reward function is expanded so the robot learns to dodge and avoid incoming hits, not just deal damage.
 - **Multi-Agent Combat:** The ultimate goal: deploying two fully trained robots into the environment for a complete, autonomous sparring match.
 - **Sequence Combat:** Agent learns to throw multiple punches in sequence.

Group G22 - Tools & Limitations

- Tools:
 - **Python (Core Language):** The foundation of the project, utilized alongside standard Deep Learning libraries (like PyTorch or Stable-Baselines3) to build the neural networks and manage the training loops.
 - **MuJoCo (Main Physics Engine):** Our primary simulation platform, chosen specifically for its accuracy in simulating complex, hard-contact dynamics.

Group G24 - Formal & Title & Goals

- Team Members:
 - Bruno Moreira <up202107143@up.pt>
 - Tiago Gonçalves <up202205081@up.pt>
 - Tomás Vinhas <up202208437@up.pt>
- Title and General Objectives - Adversarial Capture the Flag: Emergent strategies of Reinforcement Learning vs Finite State Machines
 - Create a capture the flag scenario where two agents compete, one attacks (FSM), the other defends (RL)
 - Implement an FSM heuristic-based robot to play Capture the Flag
 - Implement an RL-based approach to sabotage the FSM and cause a timeout
 - Establish comparisons and understand implications between approaches

Group G24 - Intermediate Goals

- Intermediate Goals:
 - Develop a symmetrical 3D arena in Webots
 - Configure the robots to be utilized
 - Develop the FSM for the scripted bot to play CTF perfectly
 - Implement robust pathfinding/tracking algorithms for FSM bot navigation
 - Write a supervisor node to act as the referee in Gymnasium and calculates the reward signal
 - Wrap the Webots environment in a custom gymnasium interface
 - Design and iteratively tune the dense reward function using Stable-Baselines3 (PPO)

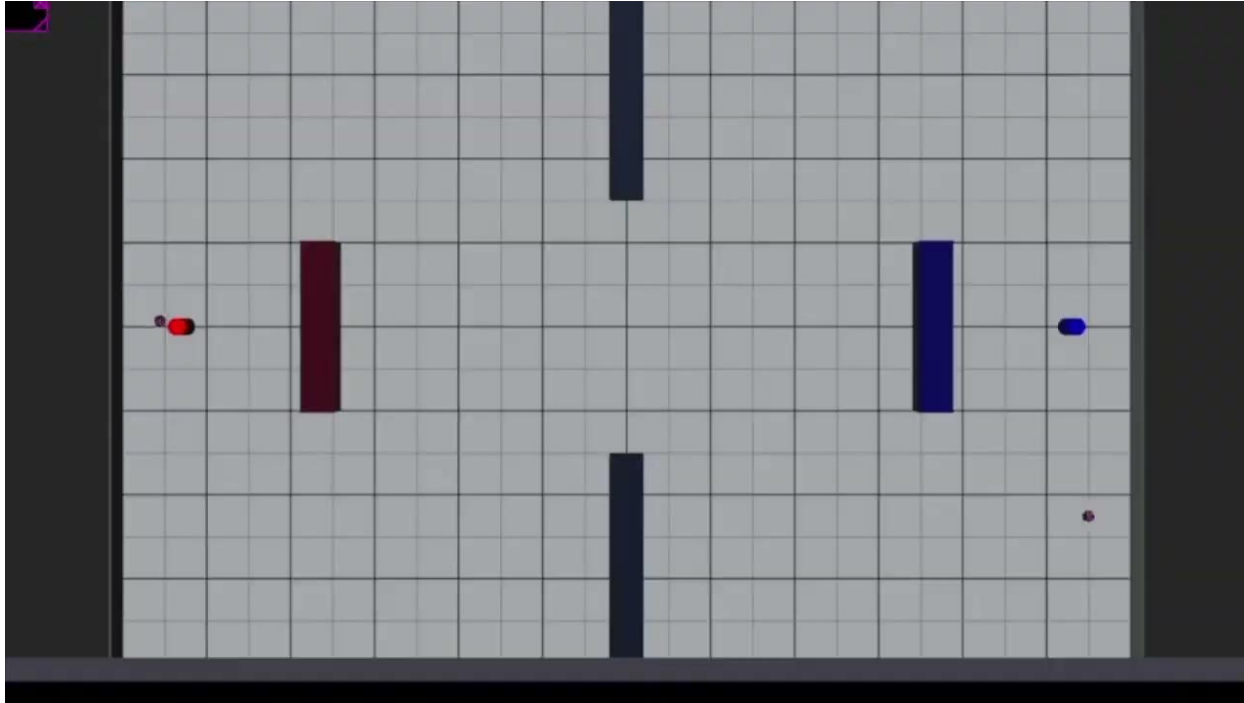
Group G24 - Course Project (Final) Goals

- Main Goals:
 - Successfully train a single-agent RL robot capable of executing complex, multi-step tasks and achieves a >50% win rate against the scripted FSM bot in sabotage
 - Compare learned vs scripted behaviour in a competitive, adversarial setting, discover “emergent strategies” that were not explicitly programmed
 - Evaluate both the models based on metrics
- Extra Merit:
 - Progressively increase arena complexity
 - Multi-Agent RL, where many agents learn by cooperating in sabotage against various FSM robots
 - Utilise visuals and heatmaps to understand RL behaviour and results
 - Create solutions robust to real-world noise and scenarios

Group G24 - Tools & Limitations

- Tools:
 - Python3 as the language
 - Webots for the physics engine and simulation
 - Stable-Baseline3 for the RL algorithm implementation
 - Gymnasium for the standard API for RL environments
 - Matplotlib for generating performance graphs and other visualizations
- Limitations:
 - Scripted model complexity
 - Training time and compute power
 - Struggling with “real world scenarios” with noise

Group G24 - Evidence of Intermediate Goals



Group G24 - Evidence of Intermediate Goals

- **All intermediate goals achieved, progress towards final presentation and extra merit:**
 - Webots robots and arena
 - Robust A* implemented with obstacle detection for the FSM robot(s)
 - Training and evaluation loops, with checkpoint brain saving, for both 1v1 and 2v2
 - Reward functions (progressively altered) designed and implemented, with the RL saboteur learning at each stage, already close to final for both 1v1 and 2v2
 - Speed (and max steps) progressively increased in learning, now both robots equal at 0.33 m/s maximum speed, started at 0.25 m/s for RL and 0.1 m/s for FSM
 - RL robot winning on 1v1 scenarios, with fully randomized spawn (behind the flag)
- **Difficulties and Future Work:**
 - Design of the reward functions and alterations of supervisor for final edge cases
 - Design of progressive scenarios that allow learning
 - Compute power (RL takes a long time to learn)
 - Localization of the fsm robot by the RL robot, potentially with noise
 - Creation of Visuals and Heatmaps for RL analysis

Group G29 - Formal & Title & Goals

- Team Members:
 - Alexandre Ramos <up202208028@up.pt>
 - Ana Filipa Geraldes <up202208030@up.pt>
 - Filipa Fidalgo <up202208039@up.pt>
- EDER: Evasive Dribbling Engine for Robotics
 - Create a Webots simulation environment where a two-wheeled robot learns to detect a ball, navigate towards it, control it, and carry or push it into a goal area within a time limit.
 - The robot must learn how to move efficiently in a dynamic environment, using reinforcement learning to adapt its behaviour to different ball positions, initial conditions, and possible obstacles.

Group G29 - Intermediate Goals

- Intermediate Goals:
 - Implement a two-wheeled robot model in Webots.
 - Implement basic robot locomotion and low-level control.
 - Create the environment, including the field, ball, and goal area.
 - Implement ball interaction logic, first without LiDAR to focus on RL.
 - Define the robot's knowledge domain, such as ball position, robot orientation, distance to goal, and obstacle information if available.
 - Define the robot action space, such as left and right wheel velocities.
 - Train an initial reinforcement learning agent to reach the ball and move it towards the goal without obstacles.
 - Monitor training progress using reward evolution and task success indicators.

Group G29 - Course Project (Final) Goals

- Main Goals:

- Train a robust RL agent to detect, approach and control the ball, pushing it towards the goal within a time limit.
- Adapt to different initial positions of the robot and ball using stable differential-drive movement.
- Optimize rewards for ball approach, control, goal progress and successful scoring.
- Evaluate performance through success rate, time to goal, ball contact rate and scoring rate.
- Analyse the learned behaviour and decision strategies.

- Extra Merit:

- Add static obstacles and a moving defender with variable speed to make the task less predictable.
- Train the robot to avoid obstacles while keeping control of the ball.
- Add a kicking or lever mechanism for stronger or upward shots.
- Compare performance in simple and complex environments.

Group G29 - Tools & Limitations

- Tools:

- Webots for robot simulation and environment design.
- Python for reinforcement learning training and environment control.
- Reinforcement learning algorithms such as PPO.
- TensorBoard or equivalent tools for monitoring training.
- Webots controllers and supervisor logic for managing episodes, resets, rewards, and environment randomization.

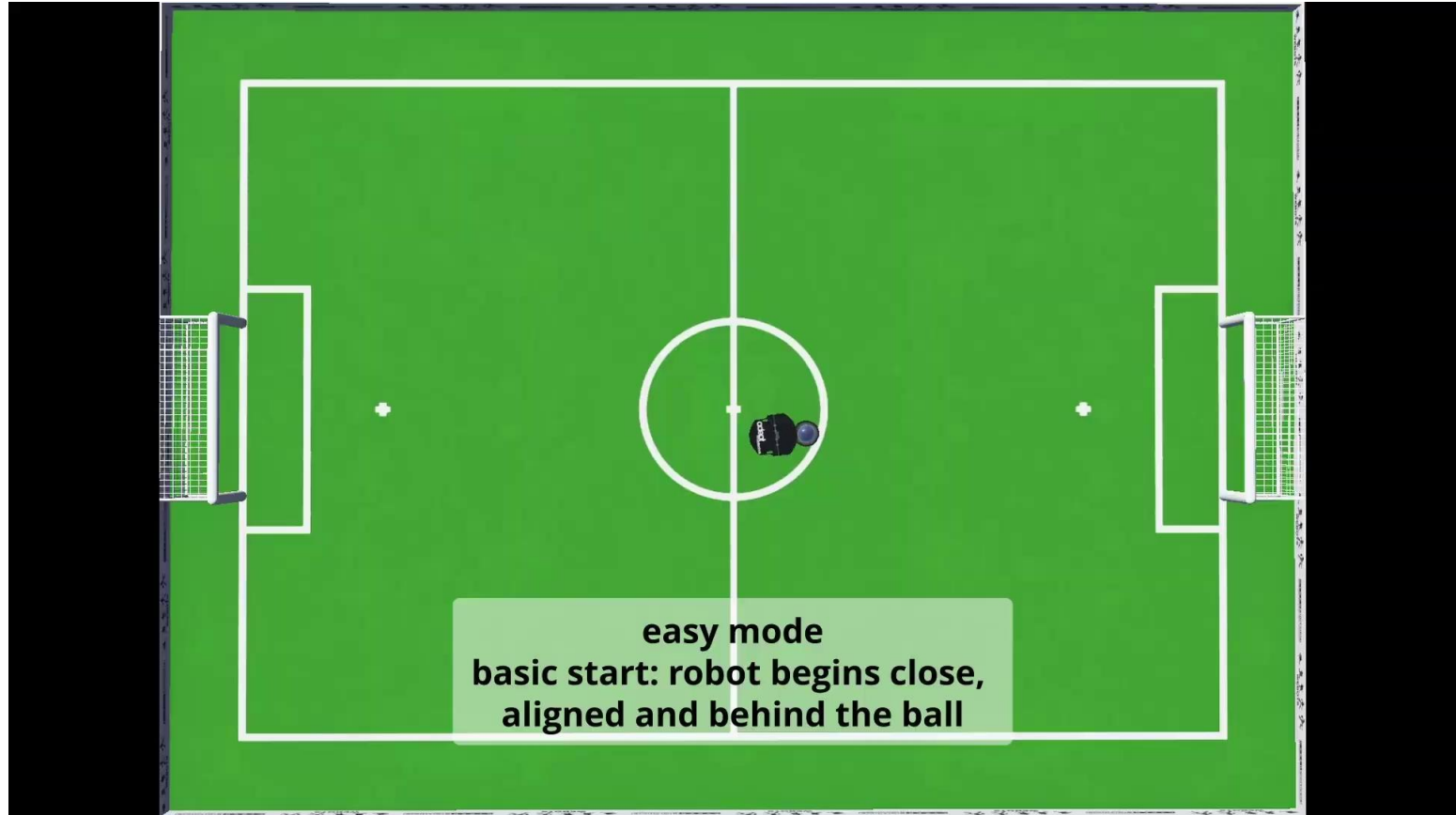
- Limitations:

- Simplified physics compared to a real robotic football scenario.
- Ball control may be difficult due to contact dynamics and wheel-based movement.
- Reward design can strongly affect training stability and final behaviour.
- Reinforcement learning training may require significant tuning and computational time.
- Perception may initially be simplified using simulated position data instead of full computer vision.
- Dynamic obstacles increase task complexity and may require longer training.

Group G29 - Evidence of intermediate goals

- Implemented and validated a Webots RL environment for the Pioneer 3DX robot.
- Developed the main RL controller with:
 - robot and ball reset logic;
 - goal detection;
 - reward shaping;
 - PPO training pipeline;
 - deterministic and stochastic evaluation.
- Achieved stable performance in the base task:
 - `EASY_RESET_PROB` = 0.95
 - Deterministic success rate: 97%
 - Stochastic success rate: 97%
- Evidence:
 - evaluation CSVs;
 - terminal logs;
 - short video demonstration of successful goal scoring.

Group G29 - Evidence of intermediate goals



Group G29 - Current limitations and next steps

- **Current Status:**
 - The robot can approach and push the ball towards the goal.
 - The agent performs consistently in controlled and moderately random initial conditions.
 - Goal detection and evaluation metrics are working correctly.
- **Main Limitation:**
 - Performance decreases when initial positions become too random.
 - Some medium/hard resets still cause the ball to move sideways or the robot to lose recovery.
- **Next Steps:**
 - Improve robustness to more random initial positions.
 - Use LiDAR to detect ball.
 - Continue testing smoother action control and ball dynamics observations.
 - Extend the environment with obstacles and a moving defender (already started).
 - Compare simple vs complex environment performance.

Group G50 - Formal & Title & Goals

- Team Members:

- João Proença <up202207835@up.pt>
- Joana Pimenta <up202206120@up.pt>
- Mihai Mărtinaş <up202512406@up.pt>



- WAVE-GUIDE: A RAG-Powered Virtual Assistant for the Waveshare UGV Orin ROS2

- **Interactive Support:** Converts the Waveshare Wiki into a real-time conversational interface for instant technical help.
- **Guided Deployment:** Streamlines complex tasks like SLAM and autonomous navigation through step-by-step guidance.
- **Command Synthesis:** Generates optimized ROS2 and Docker commands specifically formatted for the Jetson Orin.
- **Technical Accessibility:** Translates complex robotic terminology into clear, actionable development steps for all users.

Group G50 - Intermediate Goals

- Intermediate Goals:
 - **Data Extraction:** Scrape the Waveshare Wiki into a clean Markdown, strictly preserving ROS2 terminal commands and code blocks.
 - **Embedding:** Break the text into logical chunks, keeping step-by-step sequences intact before loading them into the vector database.
 - **Memory & Prompting:** Establish conversational memory and a strict system prompt to ensure the bot relies exclusively on the provided documentation.
 - **Guided Workflows:** Design prompts that force the bot to guide users through complex tasks (like SLAM) one step at a time, preventing overwhelming walls of text.
 - **Initial MVP:** Start by vectorizing just the "Quick Start" page to validate if the system can accurately answer basic setup queries.

Group G50 - Course Project (Final) Goals

- Main Goals:
 - **Interactive Documentation Chatbot:** To transform static UGV Rover Jetson Orin ROS2 manuals into a conversational partner that can reason through complex setup steps, citing specific source materials to ensure operator trust and hardware safety.
- Extra Merit:
 - Agentic Code Generation
 - Iterative Simulation Validation
 - Guardrail layer

Group G50 - Tools & Limitations

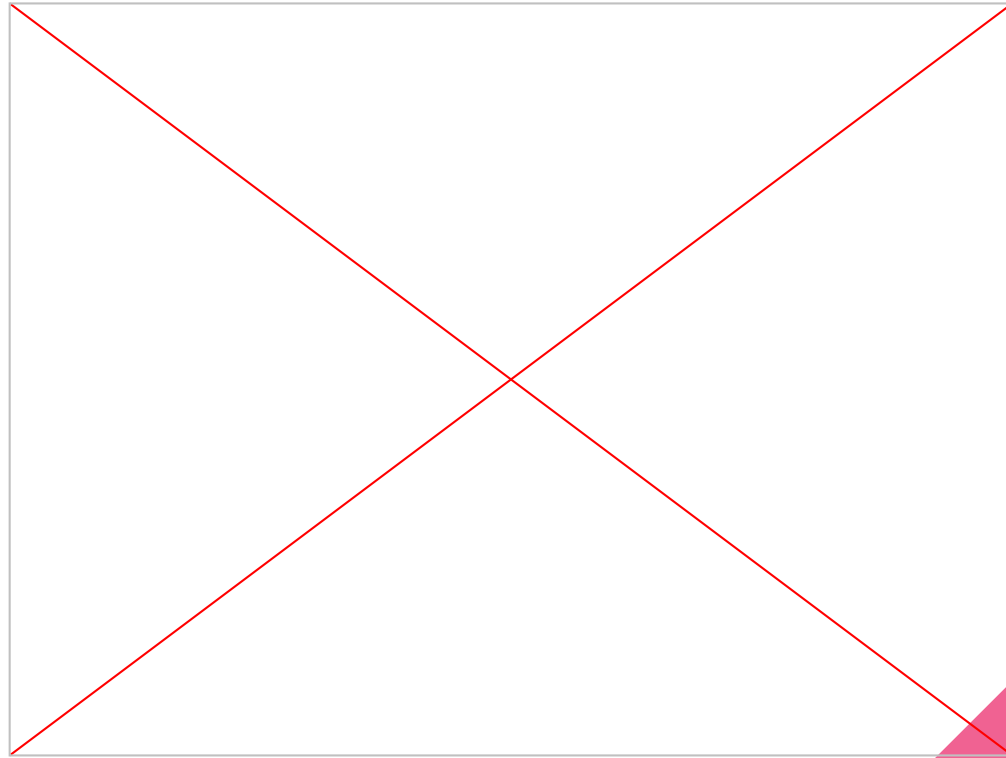
- Tools:

- Python
- Hugging Face
- Docker
- FastAPI
- Langchain/ Langgraph
- Postgres/ Pinecone
- RAGAS

- Limitations:

- Source Fidelity
- Hardware Isolation
- Static Knowledge

Group G50 - Evidence of Intermediate Goals



Group G50 - Evidence of Intermediate Goals

We finished all the intermediate goals we had planned:

- Data Extraction, Embedding, Memory & Prompting, Guided Workflows and Initial MVP

We also did:

- Guardrail layer — LLM-based off-topic rejection
- User authentication (login/register)
- Persistent chat history with auto-generated titles
- Clickable source references per answer
- Query rewriting + scope classification (routes to ROS 2, Waveshare, or both)
- Embedded the entire waveshare wiki and ros2 jazzy (excluding packages)

For the final delivery we hope to:

- **Expanded Ingestion:** Integrate core ROS 2 and Waveshare packages into the knowledge base.
- **LLM-as-a-Judge:** Automate answer scoring against a 50-question benchmark using a secondary LLM.
- **Coding Agent Evolution:** Upgrade WAVE-GUIDE from Q&A to generating ROS 2 files and configs.
- **RAGAS Evaluation:** Standardize metrics to measure retrieval and generation quality.
- Talk with colleagues using the wave Waveshare UGV Orin ROS2 to see our projects helpfulness.

Class 1 MIA03

Group G10 - Formal & Title & Goals

- Team Members:
 - Gonalo Esteves <up202203947@up.pt>
 - Inês Amorim <up202108108@up.pt>
 - Pedro Afonseca <up202205394@up.pt>
- Title and General Objectives - Autonomous Racing Car for Obstacle Avoidance with RL
 - Develop a mobile robot (car) capable of autonomous navigation on a predefined track
 - Implement obstacle detection and avoidance in real-time
 - Optimize speed while maintaining safe navigation
 - Ensure the robot completes the course in the shortest possible time

Group G10 - Intermediate Goals

- Intermediate Goals:
 - Test the robotic car platform (motors, chassis, control unit)
 - Develop basic movement control (forward, turning, speed control)
 - Develop and test obstacle avoidance strategies

Group G10 - Course Project (Final) Goals

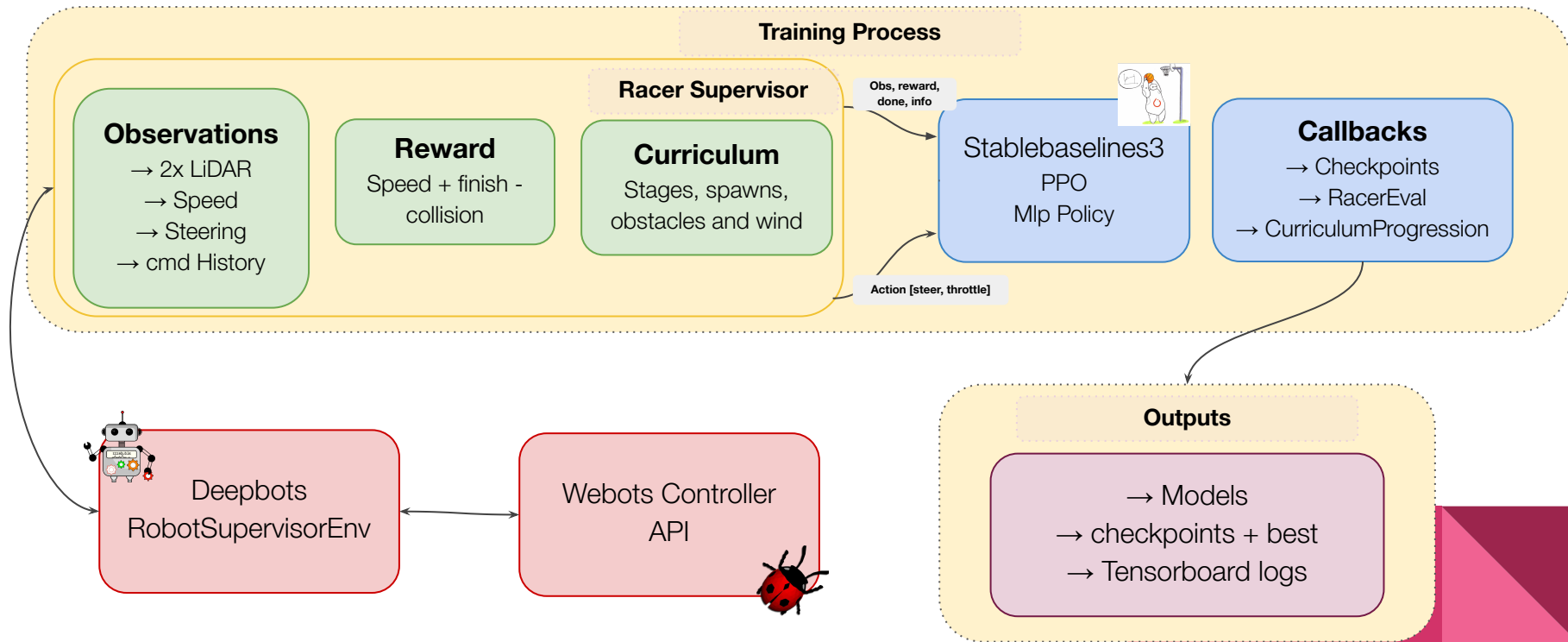
- Main Goals:
 - Perform state-of-the-art (SoA) research on autonomous navigation and obstacle avoidance
 - Achieve fully autonomous navigation in the track environment
 - Complete the full course without collisions
 - Minimize completion time through optimization of speed and path
 - Evaluate performance metrics (time, accuracy, collisions)
- Extra Merit:
 - Generalization to different tracks
 - Comparison of performance with and without wind/noise disturbances
 - Adaptive nitro activation based on track position or obstacle distance

Group G10 - Tools & Limitations

- Tools:
 - Webots simulator
 - Stable-Baselines3
 - OpenAI Gymnasium
- Limitations:
 - Computational Power
 - Noise can affect decision-making and obstacle detection accuracy
 - Controller performance may depend on careful parameter tuning
 - No real-world validation

Group G10 - Evidence of Intermediate Goals

Architecture



Group G10 - Evidence of Intermediate Goals



- **Progress**

- Designed 2 tracks for the car with different complexities
- Implemented a autonomous car robot controller trained with PPO
- Employed a curriculum learning procedure
 - The robot trains over different portions of the track at different phases and at different difficulty levels (introducing obstacles)

Group G10 - Evidence of Intermediate Goals

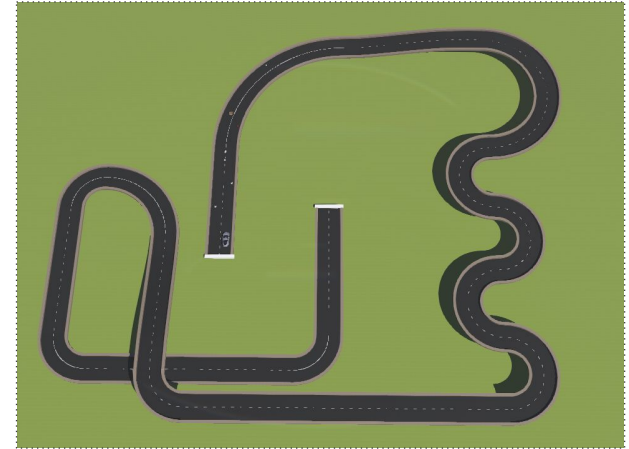
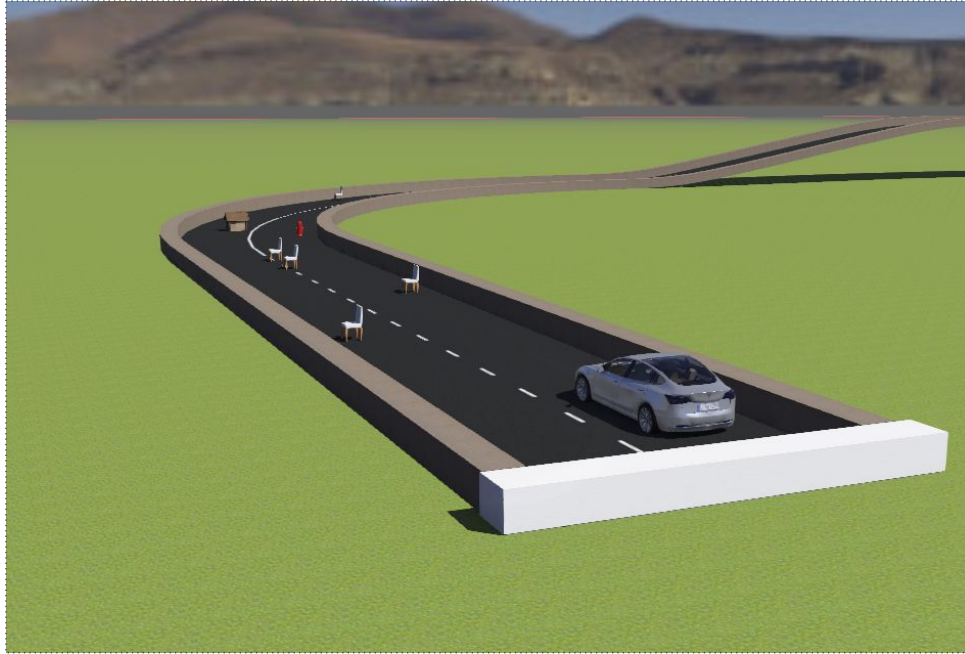


- **Difficulties**

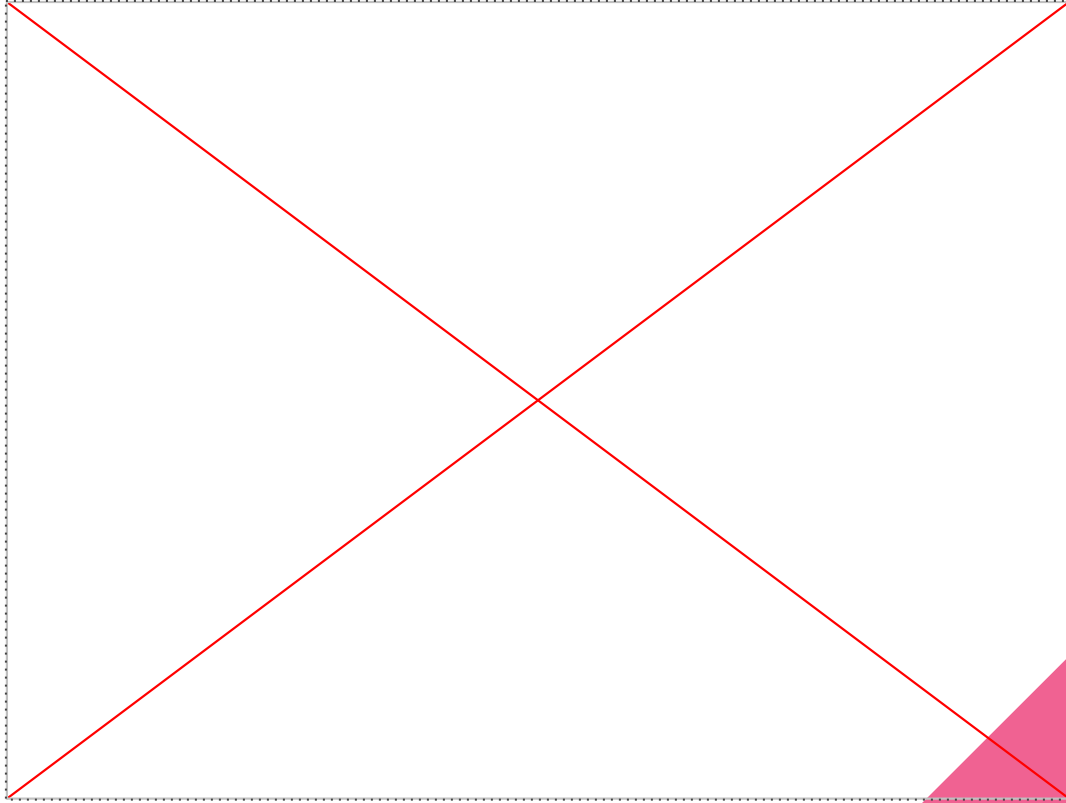
- Moving up ramps
 - Fixed by using 2 LiDars at different heights
- Oscillating vehicle leading to unstable movements.
 - Fixed by limiting the steering angle per timestep
- High velocities can make the car flip over at close curves.
 - Not fixed yet (under development)

Group G10 - Evidence of Intermediate Goals

Map Structure and Car Movement Demo



Group G10 - Evidence of Intermediate Goals



Group G14 - Formal & Title & Goals

- Team Members:
 - António Santos <up202105587@up.pt>
 - Gonçalo Dias <up202108517@up.pt>
 - Niklas Kleuser <up202511846@up.pt>
 - Paulo Silva <up202107359@up.pt>
- Title and General Objectives - Real-Time Environment Mapping and Localization with an UGV Rover
 - Implement a robust Simultaneous Localization and Mapping (SLAM) pipeline on a real-world Waveshare UGV Rover
 - Generate accurate 2D maps of unknown environments



Group G14 - Intermediate Goals

- Intermediate Goals:
 - Setup the ROS 2 & Gazebo simulation environment for testing purposes
 - Setup the physical environment and manually drive the robot
 - Research SLAM algorithms and choose the one to implement

Group G14 - Course Project (Final) Goals

- Main Goals:
 - Implement a robust Simultaneous Localization and Mapping (SLAM) pipeline on a manually-controlled real-world Waveshare UGV Rover
 - Generate 2D maps of unknown environments
- Extra Merit:
 - Autonomous-driving SLAM
 - Obstacle recognition

Group G14 - Tools & Limitations

- Tools:
 - Waveshare UGV Rover (Jetson Orion Nano)
 - ROS 2
 - Gazebo
 - Python
- Limitations:
 - Computing power for Gazebo simulations
 - Difference between real-world and simulated environments

Group G14 - Evidence of Intermediate Goals

- **Setup the physical environment and manually drive the robot**
 - Connected to the UGV Rover via SSH
 - Controlled the robot via
 - Python controllers
 - WEB interface
 - Wireless gamepad



Group G14 - Evidence of Intermediate Goals

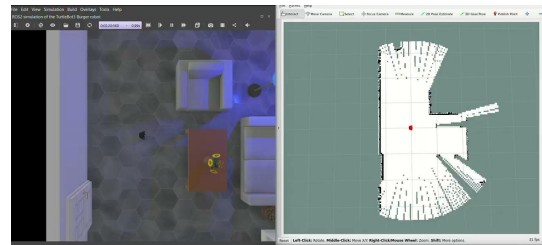
SLAM Algorithm: 2D LiDAR occupancy-grid with scan matching

- Feasible to implement from scratch
- Compatible with standard ROS 2 topics
- Suitable for both simulation and the UGV Rover

How does it work?

The mapper uses two ROS 2 topics:

- /scan: distance measurements from the LiDAR
- /odom: estimated robot movement from wheel odometry



The LiDAR scan is inserted into a 2D occupancy grid:

- cells along each LiDAR beam are marked as free
- cells where a beam hits an obstacle are marked as occupied
- the updated map is published as /map
- the position gets estimated from the initial pose + wheel odometry

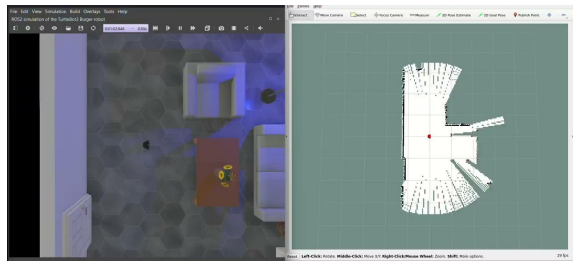
Group G14 - Evidence of Intermediate Goals

SLAM Algorithm: 2D LiDAR occupancy-grid with scan matching

In Webots, odometry is very accurate, so the map is built correctly.

On a real robot, odometry can drift because of:

- wheel slip
- uneven ground
- inaccurate wheel motion
- small sensor or timing errors



To correct errors we use **scan matching**:

- Test small pose corrections
- Match the LiDAR data to the best matching pose
- The map is updated using this pose

Group G14 - Evidence of Intermediate Goals

Next steps:

- Improve scan matching
- Prepare UGV Rover transfer
 - record real rover bags
 - transform LiDAR data if needed
 - test offline first
 - test live
- Develop and test autonomous driving controllers to lay the groundwork for autonomous SLAM (**extra merit**)

Group G15 - Formal & Title & Goals

- Team Members:
 - Dany Ferreira up202108799@up.pt
 - David Carvalho up202208654@up.pt
 - Gabriel Boia up202503471@up.pt
- Title and General Objectives - Vision-Based Gesture Control for the Waveshare UGV Rover
 - To design and implement a real-time vision-based gesture control system that enables the UGV Rover to respond to human commands in a safe, robust, and intuitive way.

Group G15 - Intermediate Goals

- Intermediate Goals:
- Check UGV Rover hardware and software environment and validate ROS2 communication.
- Verify camera access, image transport, and robot motion control.
- Design a small gesture vocabulary for rover control.
- Develop a gesture recognition pipeline using computer vision.
- Map recognized gestures to robot actions through ROS2 nodes.
- Implement safety mechanisms such as emergency stop, invalid-command rejection, and motion timeout.
- Test the system in controlled indoor scenarios and refine the recognition and control pipeline.

Group G15 - Course Project (Final) Goals

- Main Goals:
 - Demonstrate real-time gesture-based control of the UGV Rover in a real environment.
 - Achieve reliable recognition of a predefined gesture set.
 - Ensure safe operation through command filtering and stop behavior under uncertain conditions.
 - Evaluate the system using performance metrics such as recognition accuracy, response latency, command success rate, and robustness under repeated trials.
- Extra Merit:
 - Make the UGV robot execute a sequence of instructions;
 - Make the UGV robot follow a close object;

Group G15 - Tools & Limitations

- Tools:
 - Waveshare UGV Rover
 - ROS2 for communication and software integration
 - Python for rapid development
 - OpenCV for image processing
 - Media Pipe for gesture recognition
- Limitations:
 - Sensitivity to lighting and background;
 - Limited gesture vocabulary;
 - Distance and camera angle constraints;
 - Latency.

Group G16 - Formal & Title & Goals

- Team Members:
 - Angelina Moreira <up202208919@up.pt>
 - Pedro Silva <up202205150@up.pt>
 - Pedro Pereira <up201708807@up.pt>
 - Rafael Magalhães <up202206499@up.pt>
- Title and General Objectives - A Safe duckie is a smart duckie: Quacksleigh
 - Explore SotA on automated driving
 - Develop a robot capable of automatically and safely navigating a city environment
 - Experiment with real world deployment of AI driving robot

Group G16 - Intermediate Goals

- Intermediate Goals:
 - Explore SotA for relevant fields.
 - Setup simulator and real world environment.
 - Start implementing RL-based driving system.

Group G16 - Course Project (Final) Goals

- Main Goals:
 - Explore SoA on automated driving systems.
 - Implement RL-based driving system.
 - Implement Imitation Learning based driving system.
 - Apply simulated model to real-world environment.
 - Document findings and approaches.
- Extra Merit:
 - Stress test collision avoidance;
 - Experiment with pre-trained CNNs for feature extraction;

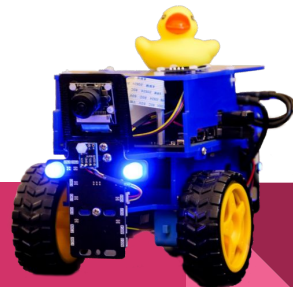
Group G16 - Tools & Limitations

- Tools:

- [FEUP's Duckie Town Lab](#)
- [Duckie Town Simulator](#)
- <https://github.com/kaland313/Duckietown-RL/>
- Python for robot/other environment controllers

- Limitations:

- Limited time-frame for real-world experiments.
- Only 1 front-facing camera makes certain maneuvers impossible such as overtaking, or driving in reverse.



Group G16 - Evidence of Intermediate Goals

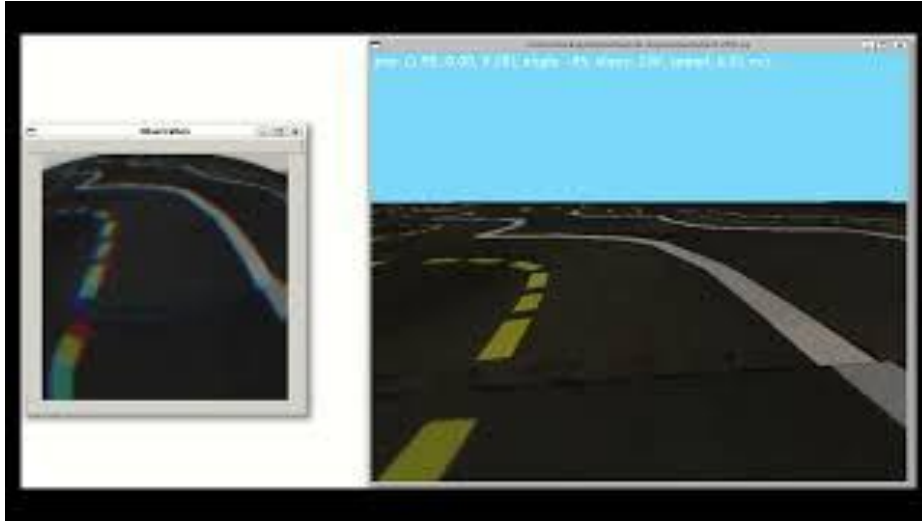
- Architectures:
 - RL: PPO
 - Vision: CNN
- Difficulties:
 - Training RL
 - Setting up the real robot

RL parameters

```
sample_batch_size: 265
train_batch_size: 4096
gamma: 0.99
lr: 5.e-5
seed: 1118
lambda: 0.95
sgd_minibatch_size: 128
vf_loss_coeff: 0.5
entropy_coeff: 0.0
clip_param: 0.2
vf_clip_param: 0.2
grad_clip: 0.5
```


Group G16 - Evidence of Intermediate Goals

Without Obstacles



With Obstacles



Group G25 - Formal & Title & Goals

- Team Members:
 - Diogo Costa <up202502576@up.pt>
 - Johnny Fernandes <up202402612@up.pt>
 - Nelson Neto <up202108117@up.pt>
- Title and General Objectives - RL-Based Autonomous Shepherd Robot for Livestock Herding
 - Implement effective herding behaviors through proximity and movement strategies
 - Build a 3D environment with realistic robot dynamics and LIDAR-based perception
 - Develop a mobile robot capable of autonomously guiding a flock of sheep into a designated target area using Reinforcement Learning

Group G25 - Intermediate Goals

- Intermediate Goals:
 - Set up the **Webots** simulation environment with an open field and target zone
 - Implement lightweight **Gymnasium**-based 2D herding environment
 - Design a **Sheep** and **Dog** robot
 - Implement a sheep flocking model for fast RL iteration
 - Validate LiDAR sensor feedback for sheep detection and distance estimation
 - Train the robot using **PPO** to successfully herd a single sheep into the goal

Group G25 - Course Project (Final) Goals

- Main Goals:
 - State-of-the-art survey on herding algorithms and multi-agent RL herding
 - Achieve fully autonomous herding of multiple sheep and a full flock into the target area
 - Quantitative evaluation: success rate, time-to-pen, flock dispersion metrics
 - Article, demo video, and final presentation
 - Curriculum Learning (scaling from 1 sheep to a flock)
- Extra Merit:
 - Optimize robot trajectory to minimize the time required to group the flock
 - Ensure zero collisions between the robot and the sheep during the task
 - Comparison of performance between **Differential Drive** and **Mecanum wheels**
 - Robustness testing under sensor noise or varying sheep speeds, configurations and parameters
 - Multi-shepherd cooperative mode: 2 dogs learn role specialization (collector vs. driver)
 - Obstacles and terrain (walls, narrow gates, corridors) the flock must be funneled through

Group G25 - Tools & Limitations

- Tools:
 - **Webots** for 3D physics simulation with ROS2 integration via webots_ros2 package
 - **Stable-Baselines3** for the PPO algorithm implementation
 - **Gymnasium** (OpenAI) for the RL environment wrapper (Lightweight 2D herding env for fast RL training)
 - **Python** as the primary programming language (sheep flocking model, reward shaping, evaluation)
- Limitations:
 - **Computational Power:** Training time might be high for complex flock behaviors
 - **Sim-to-Real Gap:** No real-world validation of the herding controller, project is simulation-only (2D + Webots 3D)
 - **Model Complexity:** Simplified sheep behavior (scripted) may not account for all biological livestock nuances

Group G25 - Intermediate Goals (Delivery)

- Intermediate Goals:
 - Set up the **Webots** simulation environment with an open field and target zone ✓
 - Implement lightweight **Gymnasium**-based 2D herding environment ✓
 - Design a **Sheep** and **Dog** robot ✓
 - Implement a sheep flocking model for fast RL iteration ✓
 - Validate LiDAR sensor feedback for sheep detection and distance estimation ✓
 - Train the robot using **PPO** to successfully herd a single sheep into the goal ✓

All intermediate goals achieved, including the a few extras:

- **Second round world**
- **360° degree LiDAR sensing**
- **Mecanum wheel preview (yet to be completed)**

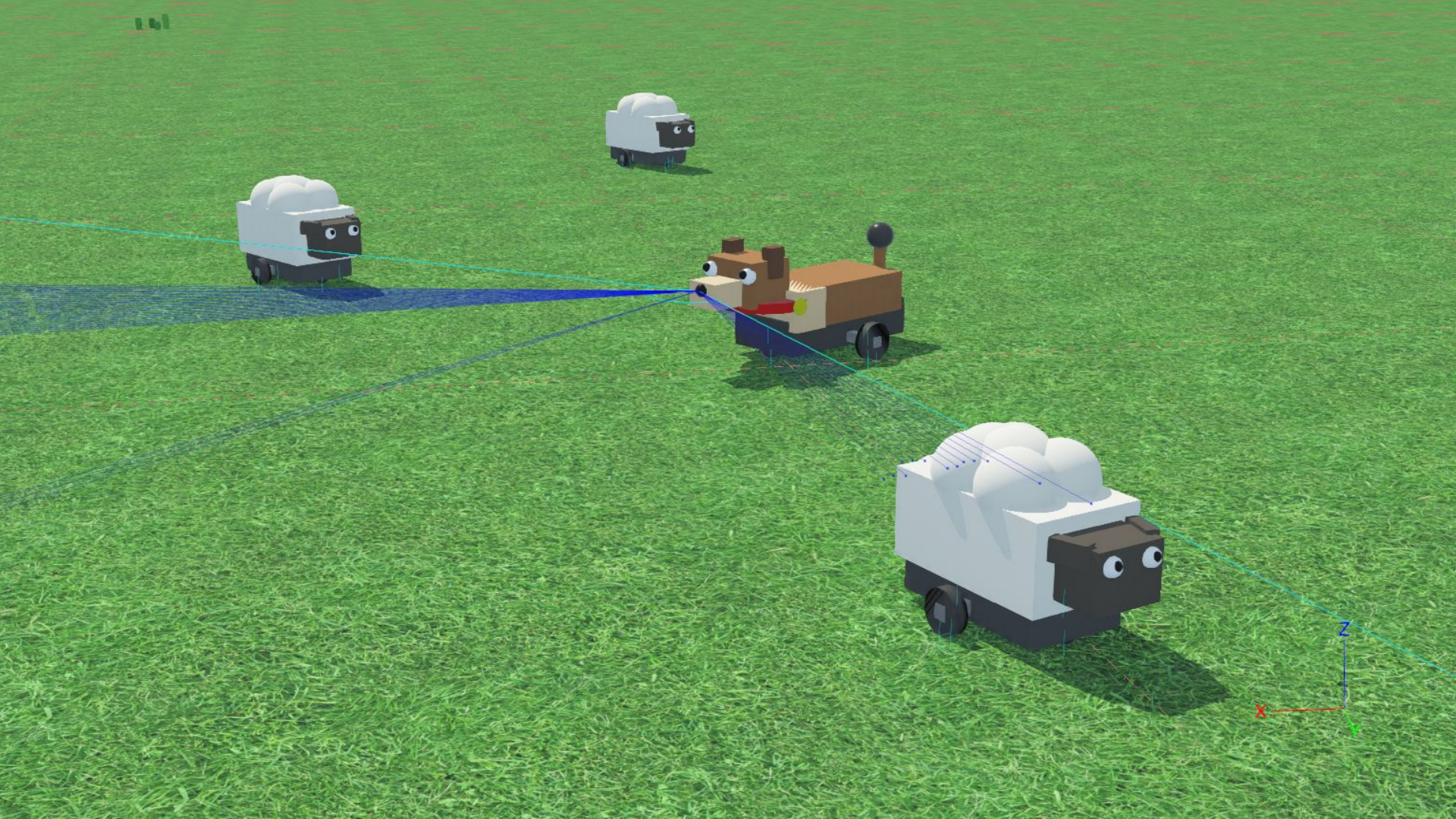
Group G25 - Intermediate Goals (Delivery)

Differential drive results

All configurations validated with 8/8 cells pass. Measured in steps:

Mode	Field, n = 5	Field, n = 10	Field_round, n = 5	Field_round, n = 10
Strömbom	7528	11620	8611	10339
Sequential	7135	16843	6019	8494
BC	11698	15079	7234	11320
RL	10039	13954	5803	9151

Key finding, RL is strictly faster than BC on every comparable cell.
Example, field n=10, **13954 vs 15079 steps (~7% faster)**.





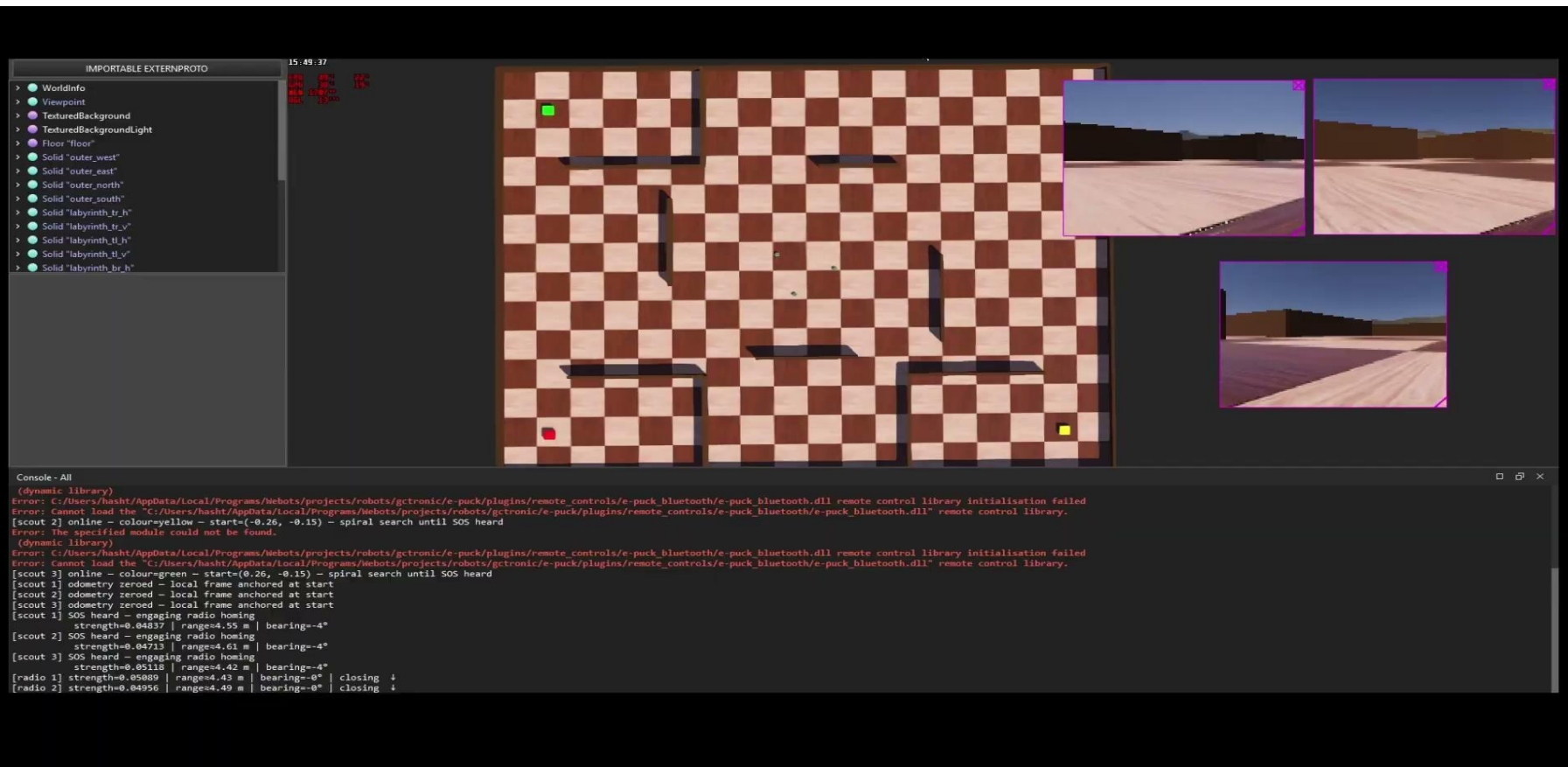
Group G28 - Formal & Title & Goals

- Team Members:
 - João Sousa <up202205238@up.pt>
 - Alejandro Gonçalves <up202205564@up.pt>
 - Francisca Mihalache <up202206022@up.pt>
 - Rafael Pacheco <up202206258@up.pt>
- Title and General Objectives - RescueBot: Safe Navigation & Pathfinding
 - Using a differential, simulated robot to act as a RescueBot, capable of “detecting” pings of help requests and navigating to them in an unfamiliar environment with obstacles.
 - The robot then outputs a map with obstruction information to reach the military team so rescue teams can help.

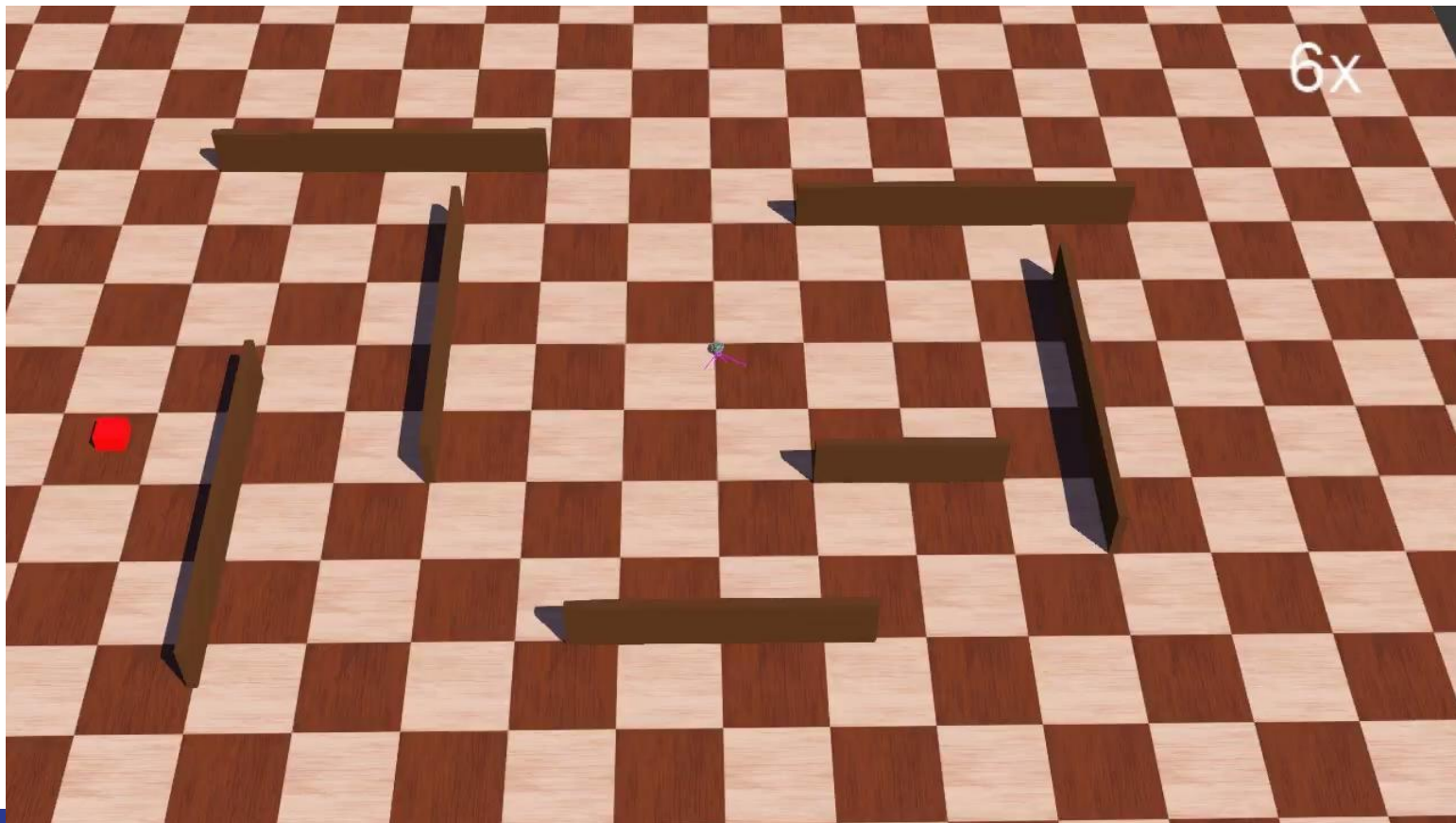
Group G28 - Intermediate Goals

- Intermediate Goals:
 - Obstacle detection and avoidance
 - Entails correctly identifying possible collision points and avoiding obstacles to ultimately reach objective: **DONE**
 - Functional pathfinding, possibly optimized through heuristics
 - Finding a path from the robot's current position to the goal: **DONE**
 - Robot reaches ping
 - Robot moves toward and arrives at the location of at least one help request: **DONE**

Group G28 - Intermediate Goals Proof (Swarm)

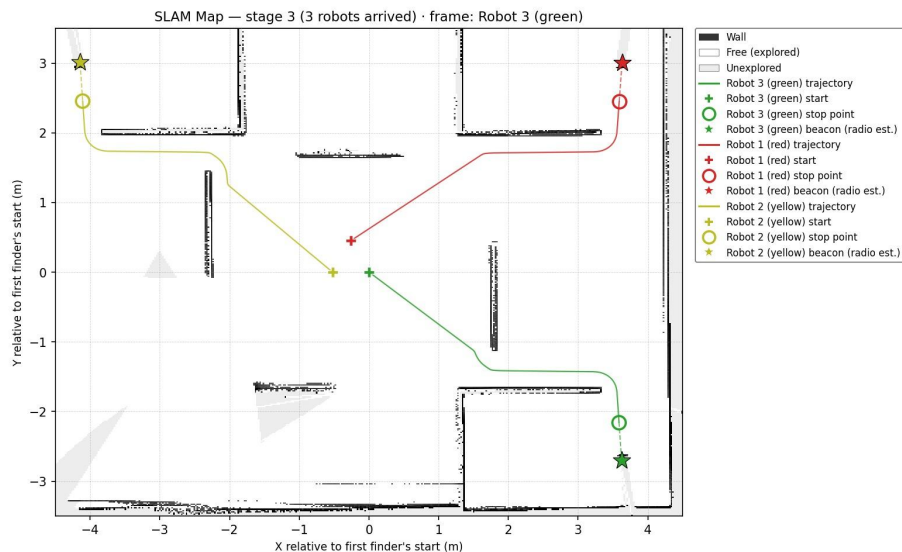


Group G28 - Intermediate Goals Proof (Optimal)

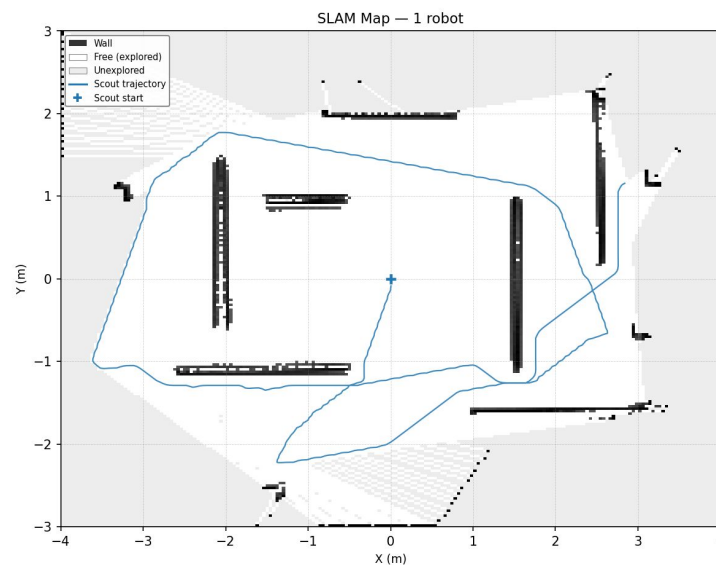


Group G28 - Intermediate Goals Proof (Maps)

Swarm



Optimal



Group G28 - Course Project (Final) Goals

- Main Goals:
 - Smart pathfinding algorithm (real-time objective distance heuristic updates) in unfamiliar environments: **DONE**
 - Mapping of obstacle locations to guarantee convergence to goal, saving a final mapped environment: **DONE**
 - Smart obstacle avoidance logic: **DONE**
 - Handling multiple help-request pings: **DONE**
- Extra Merit:
 - Using robot camera to detect type of the request for help (QR code, triage color): **DONE**
 - Potentially using a team of robots for efficiency: **DONE**
- Ambitious:
 - Combining the best parts of both approaches
 - Potentially complicating the vision task with shapes

Group G28 - Tools & Limitations

- Tools:
 - Webots: Simulation environment for testing navigation, sensors, and obstacle interaction
 - Python
 - Matplotlib, numpy, pickle, heapq
- Limitations:
 - **Time** constraints
 - **Priority handling limitations:** Conflicts between multiple high-priority pings, replanning overhead
 - **Simulation-only setup:** Results are validated in simulation (Gazebo), reducing realism.
 - **Generalization to unknown environments:** The robot operates without prior maps-> robust heuristics.
 - **Trade-off** simplicity vs performance

Group G51 - Formal & Title & Goals

- Team Members
 - Mafalda Aires <up202106550@up.pt>
- LIDAR-Aware RL Navigation in Flatland : Sensor Reduction and Feature Engineering
 - Extend the ROS 2 + Flatland RL tutorial (Filipe Almeida, FEUP) to study how the LIDAR configuration **affects PPO-based local navigation**.
 - Reduce the number of LIDAR rays and apply **feature engineering** over the raw scan, comparing learning speed, robustness, and success rate against the baseline.
 - As a complementary contribution, document the setup for macOS / Apple Silicon (UTM ARM64) and the migration of `serp\_rl` from `gym` to `gymnasium`.

Group G51 - Intermediate Goals

- Intermediate Goals:

- Setup ROS 2 Humble + Flatland + Stable-Baselines3 on Ubuntu 22.04 ARM64 (UTM/Apple Silicon).
- Reproduce the baseline `serp\_rl` tutorial (PPO agent in a 90° corridor) with the original 9-section LIDAR.
- Migrate `serp\_rl` from the deprecated `gym` API to `gymnasium` (5-tuple step, new reset signature).
- Run the baseline to convergence ($\geq 80\%$ accuracy) and record metrics: timesteps, wall-clock time, success rate.
- Add TensorBoard logging and model checkpoints to the training loop.
- Parameterise the LIDAR plugin in `serp.model.yaml` (number of rays, angular range, sampling).
- Design a set of LIDAR-derived features (min/mean per sector, left-right asymmetry, gradient).
- Define an experimental protocol: same map, same seed budget, varied LIDAR/feature configurations.

Group G51 - Course Project (Final) Goals

- Main Goals:

- Train PPO agents under at least 4 LIDAR configurations (9, 5, 3, 2 sectors) and compare convergence and success rate.
- Implement and evaluate at least 3 feature engineering strategies over the raw LIDAR readings.
- Quantify the trade-off between sensor resolution / feature richness and policy performance.
- Deliver a Mac-friendly version of the tutorial (ARM64/UTM + `gymnasium` adaptation) as a reproducibility contribution.

- Extra Merit:

- Create additional Flatland maps (curved corridor, angular corridor, Y/T junctions) and analyse generalisation.
- Sketch a hierarchical RL architecture for Y/T junctions (manager + worker, options framework) – discussion-only.
- Train on a mix of maps and assess transfer to unseen layouts.

Group G51 - Tools & Limitations

- Tools:
 - ROS 2 Humble + Flatland (2D simulator, fork by JoaoCostaIFG) on Ubuntu 22.04 ARM64.
 - Python, Stable-Baselines3 (PPO), Gymnasium for the RL environment API.
 - PyTorch CPU-only (no GPU available in VM); TensorBoard for training curves.
 - UTM virtual machine on Apple Silicon (M-series) for the development environment.
 - Git for version control; matplotlib / pandas for results analysis.
- Limitations:
 - CPU-only training in a VM — long wall-clock times per run, restricting number of seeds and configurations.
 - Flatland is a 2D simplification: no dynamics, no slip, no realistic LIDAR noise model.
 - The `serp_rl` reward function inherited from the tutorial is sparse; reward shaping is out of scope.
 - Single-task evaluation per run; cross-map transfer experiments depend on time budget.

Group G51 - Evidence: Baseline Training in Progress

